Transforming Integer Grids to Spatial Intuition: Perception, Not Planning, Governs ARC-AGI-3

Harsidak Singh Banwait
Independent Researcher
harsidak.singh.banwait.13@gmail.com

Abstract

Sample-efficient fluid intelligence requires autonomous agents that rapidly form operational models of unfamiliar interactive environments. ARC-AGI-3 evaluates this ability across uninstructed visual games under Relative Human Action Efficiency, penalizing exploratory waste quadratically. When frontier reasoning models interact with raw integer grids, discrete serialization destroys spatial topology, inducing perceptual misinduction where models squander hundreds of actions rediscovering elementary boundaries and symmetries. We introduce SARBLOH, an autonomous open-weight agent architecture structured on the premise that spatial perception, rather than planning capacity, governs interactive reasoning on ARC-AGI-3. A zero-parameter, $\mathcal{O}(HW)$ perception engine reconstructs frames in 35 ms into an object inventory with bipartite temporal tracking, a geometric briefing separating proven invariants from affordance hypotheses under the Measured–Guessed Rule, and an annotated spatial canvas. Paired with a speculative code-as-action runtime, the agent synthesizes algorithmic hypotheses rather than low-level motor primitives. Across eight structural presentations, grounded perception elevates benchmark rubric acquisition from $3/28$ to $25/28$ points ($8.3\times$) while reducing token consumption by 89.4%. On twenty-three held-out games, geometric invariants generalize unconditionally with 100% spatial accuracy, whereas single-frame affordance guesses drop to between 6% and 50%, confirming the necessity of active hypothesis verification probes. In public evaluation, Run A (briefing off; it characterizes the action interface, serving loop, and scheduler) resolved 143 of 183 levels in 8.2 hours with a median human action ratio of $0.77\times$ and 59.4% single-pass efficiency, with error decompositions attributing seventy-two percent of unearned score loss to perceptual misinduction on stalled environments; Run B (briefing on) recorded $0.68\times$ and 63.5% single-pass efficiency. Within the official nine-hour budget on a single Kaggle NVIDIA RTX PRO 6000 GPU, SARBLOH evaluated on the held-out benchmark with the full perception engine active, achieving an official hidden Kaggle score of 16.53, advancing $13.7\times$ beyond the first Milestone winner.

Code: https://github.com/Harsidak/SARBLOH

1 Introduction

Autonomous skill acquisition under epistemic uncertainty represents a foundational benchmark for artificial general intelligence. When human players encounter an unfamiliar interactive environment, they do not read procedural manuals—they visually parse the scene within hundreds of milliseconds, effortlessly abstracting embodiment, spatial boundaries, and functional affordances from core-knowledge visual priors (Spelke and Kinzler 2007). The ARC-AGI-3 benchmark (ARC Prize Foundation 2026a) formalizes this capability by evaluating agents across uninstructed, sequential game environments using Relative Human Action Efficiency (RHAE)—a metric that penalizes superfluous exploratory actions quadratically (Chollet 2019). Under the strict rules of the official offline track, candidate agents must resolve undisclosed games within a nine-hour wall-clock limit on a single NVIDIA RTX PRO 6000 GPU without external network connectivity, precluding both human intervention and unconstrained exploratory rollouts.

However, deploying language and vision-language reasoning models in this interactive regime reveals a severe representational bottleneck. While modern reasoning backbones demonstrate strong symbolic deduction when provided with clean declarative state descriptions, their execution collapses when exposed to raw 64×64 integer coordinate matrices. In preliminary experiments (Section 3), a state-of-the-art frontier model expended over 200–300 exploratory actions navigating an elementary maze serialized as raw tokens, exhausting its action budget and failing the task. In stark contrast, providing the identical model with an aligned visual rendering and geometric breakdown reduced required actions to 6–12, surpassing human baselines. This divergence demonstrates that policy deduction is rarely the primary failure mode; rather, unparsed token grids systematically destroy topological adjacency, forcing autoregressive attention to reconstruct basic spatial relations at prohibitive token and action costs.

To resolve this fundamental bottleneck, we conduct extensive empirical ablations and establish two central insights:

1. **Perceptual Misinduction**—Serializing spatial environments as discrete token matrices causes reasoning models to induce flawed topological models, triggering catastrophic quadratic action penalties during early exploratory

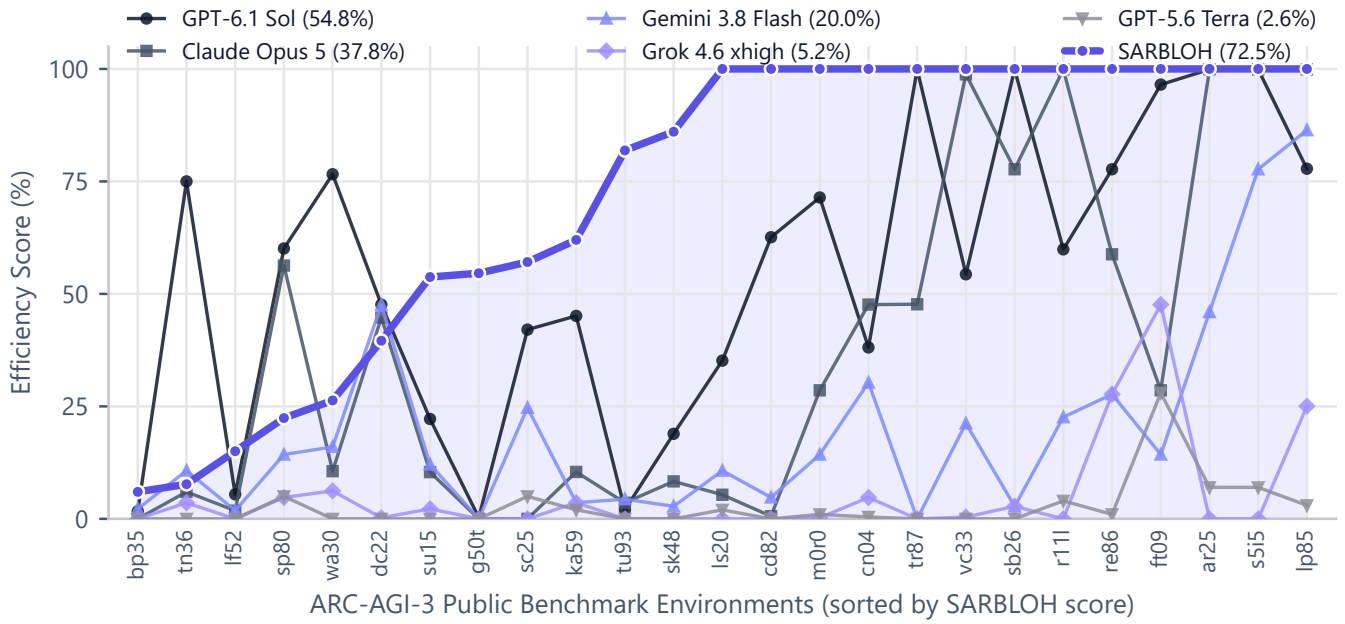


Figure 1 | Action efficiency across 25 public ARC-AGI-3 environments. SARBLOH (open-weight `Qwen3.8-Flash-Next` on one GPU) leads five frontier API baselines on 21 of 25 games (72.5% peak mean vs. 59.4% single-pass mean; 100% on 13 games). Because the strict 9-hour Kaggle ceiling precludes exhausting all 25 games in one pass, the curve reports the best score per game across runs, including isolated re-evaluations on complex environments (`bp35`).

steps.

2. **The Measured–Guessed Rule**—Geometric invariants, including lattice reachability, spatial enclosures, midpoint tethers, and dihedral symmetries under $D_4 \times \mathbb{Z}^+$, can be computed deterministically in $\mathcal{O}(HW)$ host code without training, whereas high-level affordance roles remain latent under partial observability and must be framed as probabilistic hypotheses paired with unit-action falsification probes.

Building on these insights, we introduce SARBLOH, an autonomous open-weight agent architecture engineered around a deterministic perception engine and speculative code execution. Rather than delegating visual scene decomposition to costly neural inference, our host-side perception engine executes in approximately 35 ms per frame on a single CPU core, transforming raw environment states into three synchronized modalities: an object-centric symbolic state with temporal bipartite identity tracking, a structured geometric briefing governed by the Measured–Guessed Rule, and an affordance-annotated spatial map. The agent acts exclusively through an embedded IPython execution kernel, synthesizing algorithmic state models and dispatching multi-step trajectories via a speculative execution primitive (`await play()`) that monitors environment invariants and halts immediately upon sensory prediction discrepancies.

Evaluated under identical offline hardware constraints, SARBLOH achieves state-of-the-art action efficiency across both public and undisclosed environments. Across the 25 public ARC-AGI-3 game environments, SARBLOH resolves 143 of 183 levels in 8.2 hours, discovering distinct solutions significantly faster than the first Milestone winner on identical Kaggle hardware and model weights. On the official held-out Kaggle evaluation suite, SARBLOH advanced from an initial baseline of 0.27 to a verified score of **16.53**—a 13.7× efficiency improvement over the first Milestone winner.

The primary contributions of this work are summarized as follows:

- **We identify and analyze the visual grounding bottleneck in interactive ARC-AGI-3**, formalizing the **Perceptual Misinduction** failure mode and empirically deriving **The Measured–Guessed Rule** through controlled representation ablations (§3).
- **We propose SARBLOH**, an autonomous agent featuring a zero-parameter, $\mathcal{O}(HW)$ deterministic perception engine, a speculative code-as-action planning runtime, and a monotonic append-only KV-cache architecture sustaining a 98.4% prefix-cache hit rate (§4).
- **We evaluate SARBLOH across public and official hidden benchmarks**, demonstrating a 13.7× score gain over the first Milestone winner, providing hardware-matched comparative telemetry, and detailing the failure modes that govern interactive open-weight reasoning (§5).

2 Preliminaries

The ARC-AGI-3 benchmark (ARC Prize Foundation 2026a) defines an uninstructed, discrete-time sequential decision process structured as an ordered progression of compositional levels $\{1, \ldots, n\}$. At each interaction step t, the

	Hidden Kaggle set (offline, 1 GPU, 9 h)			**Public set** (frontier APIs, unbounded)				
	SARBLOH this work	**Duck** first Milestone winner	**Reki** second Milestone winner	**Prime Agent** open harness	**Arcgentica** Symbolica	**AVO** NVIDIA	**Provider Adapters** GPT-5.5	**Provider Adapters** Gemini 3.8 Flash
Wall-clock limit	9 h	9 h	9 h	unbounded	unbounded	unbounded	unbounded	unbounded
Network access	none	none	none	yes	yes	yes	yes	yes
Protocol	peak / single pass	–	–	self-reported	self-reported	self-reported	self-reported	self-reported
Open weights	✓	✓	✓	–	–	–	–	–
Backbone	Qwen3.8-Flash-Next	Qwen 3.6 27B	Gemma-4-31B	Claude Opus 5	Claude Opus 4.6	Opus 5 / GPT-5.6	GPT-5.5	Gemini 3.8 Flash
Score	16.53	**1.21**	**0.87**	**95.5**	**36.1**†	**100.0**†	**99.99**†	≈**99.9**†

Table 1 | ARC-AGI-3 systems across two distinct evaluation regimes. The left group executes offline within the official Kaggle evaluation sandbox on a single GPU under strict hardware and nine-hour wall-clock limits without network access. The right group evaluates proprietary frontier models via unconstrained commercial APIs without standardized verification. The official hidden-set score (16.53) represents the cumulative competition metric (E, Equation (3)), which aggregates triangular level weights across multiple undisclosed games under the nine-hour ceiling, whereas Figure 1 reports the unweighted per-game efficiency percentage (S_l, Equation (2)) on public environments. †Self-reported by authors on public games without independent certification.

agent receives a discrete visual grid observation $g_t \in \{0,\dots,15\}^{H\times W}$ with spatial dimensions bounded by $H, W \leq 64$, where each coordinate denotes one of 16 categorical color indices. In addition to the visual grid, the environment exposes the episode termination status $s_t \in \{\texttt{NOT_FINISHED}, \texttt{WIN}, \texttt{GAME_OVER}\}$ alongside the count of levels resolved so far.

To interact with the environment, the agent selects an action from the action set $\mathcal{A}$, defined in Equation (1):

$$\mathcal{A} = \{\texttt{ACTION1},\dots,\texttt{ACTION5}, \texttt{ACTION6}(x,y), \texttt{ACTION7}, \texttt{RESET}\}. \tag{1}$$

In Equation (1), actions ACTION1–ACTION4 represent cardinal directional displacements, actions ACTION5 and ACTION7 denote game-specific discrete triggers, and ACTION6(x, y) specifies an arbitrary continuous spatial coordinate click parameterized by $x \in [0, W-1]$ and $y \in [0, H-1]$. Environments provide zero upfront task directives, as action semantics, controllable entity embodiment, and terminal win conditions are latent. The agent must infer these operational rules entirely from transition dynamics, where earlier levels isolate elemental mechanics and subsequent levels compose them hierarchically (Spelke and Kinzler 2007).

Benchmark performance is evaluated using *Relative Human Action Efficiency* (RHAE). For each level l, let h_l denote the upper-median human action baseline, and let a_l represent the cumulative count of physical actions committed by the candidate agent. The per-level efficiency score S_l is defined in Equation (2):

$$S_l = \min\left(1.15, \left(\frac{h_l}{a_l}\right)^2\right). \tag{2}$$

Across an entire game comprising n levels where the agent successfully completes $k \leq n$ levels, the cumulative game efficiency score E is defined in Equation (3):

$$E = \frac{1}{\sum_{l=1}^{n} l} \min\left(\sum_{l=1}^{k} l,\ \sum_{l=1}^{n} l\, S_l\right). \tag{3}$$

Equation (2) and Equation (3) establish three governing operational properties that dictate our agent architecture:

1. *Quadratic Exploration Penalty*—Because action counts enter Equation (2) inversely squared, unguided exploratory actions penalize the final score quadratically. An agent requiring twice the human baseline actions retains only 25% of the available credit.
2. *Zero-Cost Internal Computation*—Reasoning latencies and host-side execution are strictly non-penalized, and only physical actions committed to the environment register in a_l. This structure strongly favors extensive host-side geometric parsing and speculative algorithmic planning over trial-and-error exploration.
3. *Progressive Reward Scaling*—The linear level multiplier l in Equation (3) weights later compositional levels substantially higher, rewarding agents that generalize foundational mechanics induced during earlier stages.

Offline Evaluation and Resource Bounds. The official Kaggle evaluation track evaluates candidate agents on held-out, undisclosed environments within an isolated execution sandbox without external network access. Computation is strictly bounded to a single NVIDIA RTX PRO 6000 GPU (Blackwell architecture, 96 GB VRAM) and 176 GB of host RAM, subject to an inviolable nine-hour wall-clock execution ceiling for the entire evaluation suite. Furthermore, all model weights and dependencies must be open-source and

pre-packaged within the submission container, contrasting sharply against unconstrained frontier API baselines (Table 1). Under these constraints, the operational budget is governed by generation decode throughput and the length of context history that can be retained in the GPU's warm prefix cache without triggering expensive quadratic recomputations.

3 Motivation: Perception as the Primary Bottleneck

Before designing the agent architecture, we conducted controlled empirical ablations to isolate where autoregressive reasoning models expend their exploratory action allocations during interactive problem solving.

Observation geometry dictates search complexity far more than reasoning depth. We evaluated an identical frontier reasoning model (Gemini 3.1 Pro via commercial API, greedy decoding at temperature $T = 0$, evaluated in a single diagnostic trial $n = 1$ on 12 October 2026) on the opening level of the official environment `ls20`, holding the prompt template and environment state transitions strictly invariant while altering only the input observation representation (Figure 2). While an exploratory single-trial diagnostic is inherently limited in statistical variance, its primary objective was not distribution estimation, but rather isolating the catastrophic representational failure mode that occurs before any downstream policy search begins. When provided with the environment serialized as a raw 64×64 integer matrix, the model expended between 200 and 300 actions repeatedly colliding with invisible topological boundaries, exhausting the permissible budget without reaching the goal state. Recovering reachability and spatial adjacency from 4,096 serialized integers imposes an associative retrieval burden that multi-head attention mechanisms execute unreliably (Rahmanzadehgervi et al. 2024). Supplying a rendered raster image of identical board state reduced the required exploration by an order of magnitude to 12–24 actions. Augmenting this visual input with an explicit structural decomposition of terrain, HUD gauges, and color parity further consolidated the trajectory to 12 actions. Finally, enabling the model to express search routines as executable Python scripts within a sandboxed runtime cleared the level in 6 actions, achieving super-human efficiency relative to the human baseline of approximately 10 actions. Critically, over 90% of the aggregate efficiency gain resulted from structuring the perceptual representation rather than extending inference-time chain-of-thought deliberation.

Planning failure is downstream of perceptual misinduction. To decouple strategic reasoning capacity from state representation errors, we evaluated the model on a complex `ls20` level under two informational regimes: visual screenshots alone, and screenshots accompanied by an "oracle briefing" explicitly detailing sprite roles, mechanics, and geometric dependencies. Without the briefing, the agent squandered its full action budget interacting with inert visual artifacts. With the briefing, the model formulated an optimal plan and cleared the level in 6 actions. The underlying planning mechanism was functionally capable from the outset, and its operational failure was induced entirely by erroneous state perception.

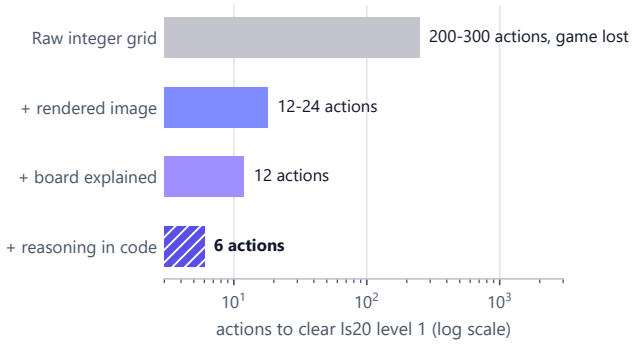


Figure 2 | Controlled ablation of input observation modalities. Action expenditure required to resolve the opening level of `ls20` across four representation tiers using an identical frontier reasoning model (Gemini 3.1 Pro, greedy decoding, single exploratory trial $n = 1$ on 12 October 2026). Serializing the environment as raw integer tokens causes budget exhaustion, whereas rendered visual rasters and programmatic geometric relations achieve super-human efficiency (6 actions versus human baseline ≈10).

Sparse delta encodings induce perceptual myopia across distant spatial structures. While transmitting only modified grid coordinates minimizes prompt token consumption, it systematically masks non-local causal couplings. In ARC-AGI-3 environments, local actuation regularly triggers distant visual consequences, such as stepping onto an activation switch in `ls20` updating a status icon across the display. Under sparse delta streams, the agent fails to observe this distributed covariance and falsely infers that its action was non-functional. Consequently, preserving full-frame topological awareness remains mandatory.

Core knowledge priors distinguish mathematical certainty from inductive hypothesis. Because oracle briefings cannot exist for undisclosed evaluation environments, the agent must extract its own representations. ARC-AGI-3 is grounded in human core knowledge priors (Spelke and Kinzler 2007), which are intrinsically geometric and topological. Properties such as lattice connectivity, bounding enclosures, midpoint tethers, and dihedral symmetries can be computed deterministically from grid coordinates without learned parameters. Conversely, operational affordances, such as agent embodiment and win conditions, remain latent until verified through physical interaction. Conflating these two epistemic categories causes models to treat transient guesses as ground truth, triggering catastrophic action loops.

Summary of Empirical Motivations.

1. **Perceptual Misinduction:** Attention mechanisms over serialized coordinate grids fail to infer topological adjacency, consuming hundreds of exploratory actions deducing baseline mechanics that are directly recoverable from visual rasters and computed geometric relations.
2. **The Measured–Guessed Dichotomy:** Geometric and topological invariants hold unconditionally across unseen environments, whereas latent operational semantics require active, single-action hypothesis verification probes.

4 Methodology

Our initial baseline, adapting the open harness of Prime Agent (Karten et al. 2026), scored between 0.00 and 0.27 on the hidden evaluation set, initiating over 350 voluntary level restarts across ten games without once resolving a goal state. The reasoning model was proficient at code synthesis, yet fundamentally incapable of spatial scene parsing. This section formalizes the governing principle of representation (Section 4.1), the deterministic perception engine that realizes it (Section 4.2), the unified programmatic action interface (Section 4.3), and the prefix-cached serving loop that fits within single-GPU hardware bounds (Section 4.4). Figure 3 illustrates an end-to-end execution turn of the complete architecture.

4.1 The Measured–Guessed Rule

We did not start from the rule; a perception lab of nine presentations, graded against an answer key written before any of them ran, produced it (Section 5.1, Figure 4).

A text-only encoder that states every cell flat scored 3 of 28. Initially, our first perception module printed the fitted lattice, a symbol per cell, the palette, connected components and object tracking. Everything it said was correct, but most of the heavy lifting was left to the model itself to identify what each cell represents. It named no agent, no goal and no gauge, and it marked the move counter of `r11l` as screen decoration.

Role guesses with tests raised the score to 20, and fusing them with computed relations raised it to 25. The Role Ledger scored every object against eight game roles and printed each top guess with its evidence and the cheapest action that would confirm or refute it (20/28, about 380 tokens). The Fused Briefing added the relations that code computes exactly—reachability, enclosure, midpoint tethers, shape equivalence—as a separate tier above the guesses (25/28, about 370 tokens). It scored $8.3\times$ the Legacy Encoder with 89% fewer tokens. Dumping raw 64×64 ASCII grids alongside per-cell component tables forces the tokenizer to spend $\approx 3{,}500$ tokens per frame encoding repetitive background coordinates; by abstracting connected components on the host, moving spatial layout into the visual Blueprint raster, and emitting only non-redundant geometric relations in text, the Fused Briefing compresses prompt footprint by 89.4% (to 370 tokens) while increasing prefix-cache capacity by $9.5\times$.

Held-out games separated the two kinds of statement. We subsequently evaluated cross-distribution generalization across 23 held-out official game environments excluded during engine prototyping. Grid reconstruction achieved 100% cell accuracy across all games, and morphological segmentation reliably preserved entity boundaries and spatial tracking without degradation. Whereas topological and geometric invariants hold unconditionally by mathematical construction, high-level semantic roles (such as agent embodiment or interactive affordances) inherently possess visual ambiguity prior to actuation. Consequently, the engine treats role labels not as brittle, hardcoded classifications, but as provisional inductive hypotheses paired with minimal unit-action verification probes. This decoupling ensures that any transient misclassification of initial sprite affordances is immediately resolved upon the first interaction step, while preserving complete coordinate, topological, and geometric stability across the entire scene.

The rule. *State what can be measured as fact; state what must be inferred as a guess that carries a one-action test.* We term this the **Measured–Guessed Rule**. A measured relation is computed from the grid by code with no learned parameters, so it is true by construction and the model need not check it. A role cannot be read from one frame, and a role stated as fact plants *perceptual misinduction*—a wrong belief about what an object is or does—that the model then spends actions on. A role stated as a guess with a test costs at most one action to settle, and under RHAE, where cost grows with the square of the action count, that cap is the point.

The rest of this section applies the rule five times. View 1 states object identity across frames as fact (Section 4.2). View 2 states geometric relations as fact. The role scorer states roles as guesses, each with its test (Table 2). View 3 draws both on one map and marks every guessed role with a question mark. Finally, `play()` treats the agent's own world model as a guess and checks each of its predictions against the next frame (Section 4.3).

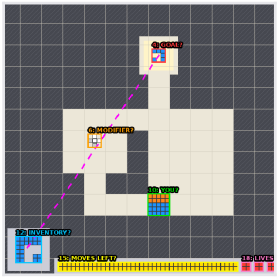
4.2 Perception: From Pixels to Stated Facts

The engine is the Fused Briefing rewritten to work on the grid inside the agent, with host-side NumPy and Pillow and no learned parameters. On 225 real frames (the first frame and eight random actions on each of 25 offline games) it raised no errors; the briefing took a median of 42 ms (maximum 0.38 s) and the picture 40 ms. Each frame yields three views that share one object-identity space (Figure 5). Algorithm 1 gives the whole mapping from grid to views in one place; reading it shows that every learned or heuristic step (role scoring) comes after every exact one.

View 1: re-segmenting each frame renamed objects, so identity is tracked, not recomputed. The perception engine identifies the background canvas as the modal color index across the frame and extracts all remaining four-connected monochromatic regions as discrete entities. Because standard morphological segmentation re-indexes components in raster reading order at every step, dynamic entities moving across the board would continually swap indices,

(a) map before

level 1, sent with (b)



(b) briefing, engine output (excerpt)

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box that is NOT plain floor;
    the floor reaches the box's bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
  ...
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press one arrow key: it should
    move one tile and the gauge should drop
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take any action and check it
    shrinks by one unit
  ...
```

(c) the agent's ipython cell at turn 7, its first action in the level

```
r = await play(["1"])
print(r.lines)
print(r.stopped, r.surprise)
new = r.obs
d = frame_diff(obs.board, new.board)
print(d)
```

(d) cell output (excerpt)

```
play: 1 of 1 moves done
#0 UP: 50 cells changed at r40-49 c34-38; the HUD changed (2 cells)
frame_diff: 0 (10 cells) moved by [-5, 0]
frame_diff: b (15 cells) moved by [-5, 0]
[state after step #0] level 1 of 7 (0 done) | 1 actions spent (1 this level),
    1999 left
```

(e) map after

obj 10 one tile up

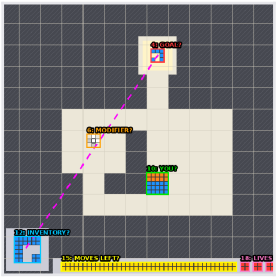


Figure 3 | One real turn, end to end (`ls20`, level 1, Run B, briefing on, Section 5.2; transcript and frames as logged). (a) The map the engine drew for the opening frame. (b) An excerpt of the briefing sent with it: measured facts first, then guesses, each with a one-action test. (c) The agent's first action in the level, written as Python at turn 7 after six turns spent only reading `observe()`: one key press, the test that the briefing proposed for YOU. (d) What the kernel printed: the player (orange and blue, 25 cells) moved one tile of 5 cells up and the HUD lost two cells. (e) The map after the move. One action confirmed two guesses, obj 10 = YOU and obj 15 = MOVES LEFT.

Algorithm 1: PERCEIVE: one frame to the briefing, the map and the object table

Input: grid g_t ($H \times W$, colours 0–15); legal actions $\mathcal{A}$; the level's objects and lattice from the previous frame
Output: briefing B, map M, object table O, change line δ

1. Objects and their ids (View 1)

$O \leftarrow$ the 4-connected one-colour regions of g_t, background (most common colour) excluded
foreach rule in order: (i) same colour, shape, place; (ii) same colour, shape, nearest; (iii) same shape, place; (iv) same colour, most shared cells **do**
  give each still-unmatched object the id of an unmatched previous object that the rule pairs it with
give every object left unmatched a new id ▷ *rule (v)*
$\delta \leftarrow$ the cells that changed since the last frame; changes within 3 cells of the edge are reported as HUD

2. Things on the screen

$\mathcal{T} \leftarrow$ the colours that cover more than 8% of g_t ▷ *terrain*
$L \leftarrow$ the other cells grouped into things: lines, bars, multi-colour composites, icons, dotted figures
mark each thing in L as HUD if it is a bar or touches the screen edge, and name it by the id that owns most of its cells
$e_{\text{you}} \leftarrow$ the largest multi-colour thing of ≥ 9 cells off the HUD, squares first ▷ *likely agent*
$\Lambda \leftarrow$ the side of the largest multi-colour square whose grid matches $> 75\%$ of the floor's edges; else the previous lattice, if it still fits

3. Measured facts: computed by code, stated as fact

$F \leftarrow$ reach (BFS from e_{you} on Λ), enclosures, midpoint tethers, alignments, look-alikes under $D_4 \times \mathbb{Z}^+$, copies, lopsided colours

4. Guessed roles: scored, stated as guesses

foreach thing $e \in L$ that is not a line **do**
  add the fixed evidence points of the eight roles (Table 2) that e's shape, place, relations in F and $\mathcal{A}$ earn
  $\hat{\rho}_e \leftarrow$ the role with the most points; $p_e \leftarrow$ its points / (all of e's points + 1)

5. Assemble within $K = 20$ lines

keep up to $\lfloor K/3 \rfloor$ lines for guesses, each with its one-action test
share the other lines among the fact kinds, one line per kind in turn; give guesses any room left over
$B \leftarrow$ the chosen facts, then the guesses, then a count of what was left out ("`+N more facts`")
$M \leftarrow$ a map of L: ids, guessed roles marked "?", look-alike threads, tether rings, HUD panel
return B, M, O, δ

(a) Legacy Encoder, 3/28
text only: palette, grid, objects; ~3,500 tokens

```
PALETTE
  '.' darkgray  rgb=[52, 56, 56] cells=2609 (terrain)
  '=' gray      rgb=[104, 104, 108] cells=892 (terrain)
  'K' black     rgb=[0, 4, 8] cells=439 (terrain)
  'Y' yellow    rgb=[256, 220, 8] cells=84 (object)
  'B' blue      rgb=[32, 148, 256] cells=45 (object)
  'R' red       rgb=[248, 64, 56] cells=12 (object)
  'O' orange    rgb=[256, 136, 32] cells=10 (object)
  'W' white     rgb=[256, 256, 256] cells=3 (object)
  '#' lightgray rgb=[204, 204, 208] cells=2 (object)
     00000000001111111112222222222333333333344444444455555555556666
     01234567890123456789012345678901234567890123456789012345678901234
 0  KKKK.............................
 1  KKKK.............................
 2  KKKK.............................
 3  KKKK.............................
 4  KKKK.............................
 5  KKKK.............................
 6  KKKK.............................
 7  KKKK.............................
 8  KKKK.............................=======
 9  KKKK.............................=KKKKKK
10  KKKK.............................=KKKKKK
11  KKKK.............................=KKBBBK
12  KKKK.............................=KKKKBK
13  KKKK.............................=KKBKBK
14  KKKK.............................=KKKKKK
15  KKKK.............................=KKKKKK
16  KKKK.............................=======
17  KKKK.............................=====
18  KKKK.............................=====
19  KKKK.............................=====
20  KKKK.............................=====
21  KKKK.............................=====
 ... (183 lines in all)
```

(b) Role Ledger, 20/28
roles + tests; ~380 tokens



(c) Fused Briefing, 25/28
map, MEASURED facts, GUESSES; ~370 tokens

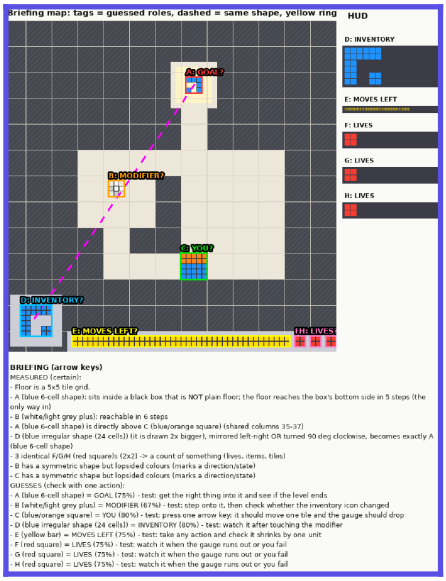


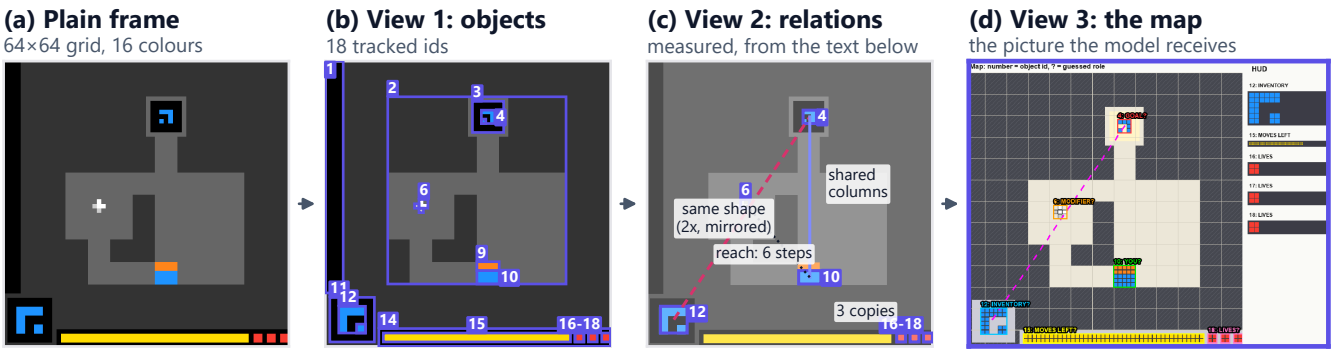
Figure 4 | Three of the nine lab presentations on the opening frame of `ls20`, with rubric score (of 28) and text cost. (a) The Legacy Encoder, text only (excerpt of 183 lines, grid rows cut at 44 columns). (b) The Role Ledger: a role map and, below it, each guess with its evidence and a one-action test. (c) The Fused Briefing, the presentation we kept: a schematic map with object tags, shape threads and tethers, then MEASURED facts and GUESSES as separate tiers. The other six presentations are in Figure 12.

causing synthesized Python scripts to silently reference incorrect objects. To maintain temporal consistency, SARBLOH anchors object identifiers on the initial frame of each level and preserves identity across transitions through a multi-stage bipartite matching algorithm. This procedure runs five passes in order: objects unchanged in colour, shape and place; objects moved with the same colour and shape (the nearest is taken); objects recoloured in the same shape and place; objects reshaped in the same colour (the one sharing most cells is taken); and, last, fresh identifiers for objects that match nothing (Algorithm 1, stage 1). As a result, each identifier reliably tracks the same physical entity throughout an entire level. In addition to the tracked object table, the spatial state is serialized as an alphanumeric coordinate grid and paired with a differential change ledger detailing the bounding box and cell count modified by each transition. Furthermore, environmental modifications occurring within the three-cell peripheral screen boundary are isolated as heads-up display telemetry, preventing passive counter decrements or timer ticks from being misinterpreted as operational task progress.

View 2: a picture does not state relations, so the engine computes them. Whether an object can be reached, how many steps it takes, whether a hollow fits another object, or whether two shapes are the same up to rotation cannot be read reliably off a rendered frame (Rahmanzadehgervi et al. 2024). The engine first splits the frame into *terrain*—colours that cover more than 8% of the frame—and things (corridors, edge gauges, multi-colour assemblies joined by union-find over 8-connected components, panel icons and dotted figures). It then computes seven kinds of measured fact:

- **Reachability**: breadth-first search over the detected *lattice*—the tile grid the player moves on—gives step counts from the likely agent to every target ("`obj 6: reachable in 6 steps`").
- **Enclosure**: closed outlines around empty space, matched against objects of the same size.
- **Midpoint tethers**: an object held at the midpoint of two others by thin lines, so moving one end moves it ("`obj 25 held at midpoint of obj 16 and 36`").
- **Alignment**: shared rows or columns between objects.
- **Dihedral symmetry**: equality of two shapes under the eight rotations and mirrorings of the square and integer scaling ($D_4 \times \mathbb{Z}^+$), as in "`obj 12 (drawn 2x bigger), mirrored left-right OR turned 90 deg clockwise, becomes exactly obj 4`".
- **Copies**: identical shapes counted as a set (lives, items).
- **Lopsided colouring**: a symmetric shape with asymmetric colours, which usually marks a direction or a state.

In `ls20`, the symmetry relation connects the HUD icon to the shape inside the goal enclosure, while in `r11l`, tether detection identifies the disc suspended between two handles. The perception lab confirmed that both relationships were



(e) The briefing: sent as text in the same message as (d), word for word

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box that is NOT plain floor; the floor reaches the box's bottom side in 5 steps (the
  only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
- obj 4 (blue 6-cell shape) is directly above obj 10 (blue/orange square) (shared columns 35-37)
- obj 12 (blue irregular shape (24 cells)) (drawn 2x bigger), mirrored left-right OR turned 90 deg clockwise, becomes exactly obj 4
  (blue 6-cell shape)
- 3 identical red square (2x2): obj 16, 17, 18 -> a count of something (lives, items, tiles)
- obj 6 (white/light grey plus) has a symmetric shape but lopsided colours (marks a direction/state)
- obj 10 (blue/orange square) has a symmetric shape but lopsided colours (marks a direction/state)
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press one arrow key: it should move one tile and the gauge should drop
- obj 12 (blue irregular shape (24 cells)) = INVENTORY (80%) - test: watch it after touching the modifier
- obj 4 (blue 6-cell shape) = GOAL (75%) - test: get the right thing into it and see if the level ends
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take any action and check it shrinks by one unit
- obj 16, 17, 18 (red square) = LIVES (75%) - test: watch it when the gauge runs out or you fail
- obj 6 (white/light grey plus) = MODIFIER (67%) - test: step onto it, then check whether the inventory icon changed
```

Figure 5 | One real frame (the opening frame of `ls20`) through the perception engine. (a) The rendered 64×64 frame. (b) View 1: the tracked objects with their ids; one-cell parts and the three HUD copies are outlined but not tagged one by one. (c) View 2: four of the measured relations from (e), drawn as straight lines between the objects they relate (a line is not a path). (d) View 3: the map the model receives: floor in light tones, walls hatched, the HUD moved to a side panel, objects tagged `id: ROLE?`, look-alike shapes joined by a dashed thread. (e) The briefing for the same frame, word for word, sent as text with the map. The model receives (d) and (e); (b) and (c) are drawn here from the engine's object table and briefing to show what each view contributes.

discoverable exclusively through these geometric computations (Section 5.1).

Proportional line quotas prevent relation flooding. Because pairwise geometric relations like coordinate alignments expand combinatorially in dense scenes (e.g., 792 alignment pairs in `bp35`), an unweighted global line cap allows high-frequency alignments to crowd out vital invariants such as symmetries and affordance hypotheses. To maintain structural balance, the briefing scheduler enforces proportional line quotas across invariant categories and reserves up to one-third of the display for candidate role hypotheses, summarizing surplus relations as aggregate counts rather than truncating them (detailed in Appendix C).

Roles: the engine guesses what objects are for, and says how to check. Non-terrain objects are scored against eight roles with fixed evidence points from game-design priors (Table 2); a role's confidence is its score over the object's total score plus one. Each guess is printed with its one-action test, so a wrong guess costs one action instead of a level.

The briefing shows at most 20 fact and guess lines in the deployed configuration: up to a third for guesses, the rest shared between the fact kinds, with facts about the guessed agent and goal first. The remainder is counted ("`+N more facts`") and stays readable in code through `observe().briefing`. Full briefings for the two lab frames are in Appendix B.3.

View 3: words go in the text, not in the pixels. A small vision encoder reads rendered text unreliably, and a sentence drawn into the image is a sentence the model must transcribe before it can use it. The map therefore carries only short tags (`10: YOU?`) that repeat ids from the text, while every sentence travels as text. It draws structure (floor in light tones, walls hatched, the lattice as fine lines, look-alike shapes joined by dashed threads, tethers marked with rings and the HUD magnified in a side panel (Figure 5d)).

Observation schedule. The full view (briefing, letter grid and map) is sent at the start of a level, after a context drain, and when the agent calls `observe()` after acting, at most once per state. After other actions the agent gets the change ledger, a letter crop around the changed cells with a 3-cell

Role	Evidence (scored)	One-action test
YOU	Compact, multi-coloured, unique, on the lattice	Press one arrow: it moves one tile
GOAL	Hollow box; shape repeated in the HUD	Bring the matching thing into it
MOVES LEFT	Long bar at the screen edge	Any action: it shrinks one unit
LIVES	Identical copies in the HUD	Watch it on a failure or reset
INVENTORY	HUD icon matching a world object	Watch it after the modifier
MODIFIER	Small symbol, lopsided colours	Step on it: the inventory changes
HANDLE	Tether end; on/off colour pair	Click it: something jumps
CARRIED THING	Held between two tether ends	Move a handle: it follows

Table 2 | Role guesses. Each is printed with its confidence and a test that settles it in one action. On held-out games these guesses were right about half the time or less (Table 4), which is why they are never stated as facts.

margin, and the new map. The object table stays on the host and is read in code (`observe().objects`).

4.3 The Action Interface

Separate tools for code and for actions split one plan across many turns. In our early builds the agent had one tool to run Python and another to submit moves. In the traces of those builds it modelled the game in one tool, left it to send a single move with the other, and came back to re-derive its state, so one plan cost many turns and every turn cost a model call. We merged the two: the agent acts only from inside its Python kernel, in the CodeAct style (Wang et al. 2024), with

```
r = await play(path, predict=f)
```

where `path` is up to 100 moves and `predict` is an optional world model $\hat{f}(s, a) \to \hat{s}$ written by the agent. It is the only change recorded for the build that moved the hidden-set score from 5.14 to 5.65. One cell can now model a level, search it and play the whole path (Figure 3c).

The agent's world model is a guess, so `play()` checks it instead of trusting it. Algorithm 2 defines the execution contract between the planned trajectory and the environment, specifying the exact conditions that govern path termination. An inaccurate state prediction does not interrupt execution. Instead, `play()` logs the first action whose predicted state differs from the true grid outside the HUD, records the mismatched coordinates, and continues along the trajectory. The path halts only when a level or game terminates, when consecutive identical actions collide with an obstacle without altering a single pixel, or when the revisit guard identifies a board state already visited during the current attempt. The loop never stalls to verify an operational rule, ensuring that any inaccuracies in the internal world model are resolved immediately upon receiving the next frame. Two of the stops are switches. The revisit guard was off in Run B (briefing on), and the replay guard, which refuses before any move a path that repeats a lost try move for move from the same first board, was on; the saved configuration of Run A (briefing off) records neither switch.

Algorithm 2: PLAY: sending a planned path and checking the agent's predictions

Input: path of moves $m_1, \ldots, m_K$; optional world model $\hat{f}$ (the agent's `step()`); current grid g_t
Output: one change line per move sent, the first surprise (or none), the last grid

if $K > 100$ **then** refuse the whole path; nothing is spent
if replay guard on **and** the path repeats, from the same first board, a try that ended in game over **then** refuse it
surprise $\leftarrow$ none
for $i \leftarrow 1$ **to** K **do**
  if $\hat{f}$ given **then**
    $\hat{g}_{t+1} \leftarrow \hat{f}(g_t, m_i)$ ▷ *predict before acting*
    if $\hat{f}$ raised an error **then** stop; m_i is not sent
  send m_i; read the new grid g_{t+1} ▷ *the move is spent*
  if $\hat{f}$ given **and** no surprise yet **and** $\hat{g}_{t+1}$ differs from g_{t+1} outside the HUD **then**
    surprise $\leftarrow (i,\ m_i,$ the differing cells$)$
    ▷ *reported; the path goes on*
  if the level was won **or** the game is over **then** stop
  if m_i changed nothing **and** the move before it was m_i and also changed nothing **then** stop (a wall hit twice)
  if revisit guard on **and** g_{t+1} was already seen in this try **then** stop
  $t \leftarrow t + 1$
return the change lines, *surprise*, g_t

Revisit guard behavior under live deployment. The revisit guard terminates path execution whenever the non-HUD board configuration repeats a state encountered earlier in the attempt. In counterfactual analysis over 9,913 actions, the guard identified 2,331 duplicate states (with a minor false-positive cost of 6 winning transitions). Under live deployment, when interrupted by the guard, reasoning models compensated by increasing single-action invocations from 8.9% to 17.8% and dispatching subsequent moves in the same cell, shifting completed levels marginally from 50 to 52 out of 97. We retain the guard as a configurable safeguard while noting that visible execution constraints can induce compensatory behaviors in reasoning backbones.

4.4 Serving Inside the Envelope

Append-only context invariant preserves prefix-cache reuse. We deploy our 125B MoE backbone quantized to W4A16 using AutoRound (Cheng et al. 2023) and served via SGLang (Zheng et al. 2023) with EAGLE speculative decoding (Li et al. 2024). The model architecture mixes Gated DeltaNet linear-recurrent layers with attention (Yang, Kautz, and Hatamizadeh 2024). Its recurrent state cannot be rewound, so the server can resume only from the state saved at the end of an earlier request, and the prefix cache matches

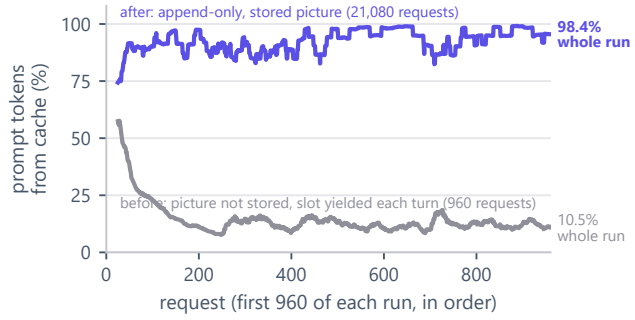


Figure 6 | Share of each request's prompt tokens served from the prefix cache (moving mean over 25 requests), for the first 960 requests of two Qwen3.8-Flash-Next runs on SGLang. Before (15 games): the picture is sent as an unstored last message and the GPU slot changes hands every turn, so each request rewrites the end of the one before. After (Run A, 25 games): every request is the previous request plus its reply, and a game keeps its slot for 20 requests. Labels on the right give the whole run.

nothing past the first modified token. To guarantee cache reuse, we enforce an *append-only* invariant: every request is the previous request plus its reply and new messages, word for word. Visual maps are retained across turns, reasoning remains in history, and working memory updates append as new lines. A game retains its GPU slot for 20 requests before scheduling handoff. Across our full benchmark run, 98.4% of 1.78 B prompt tokens were served directly from cache, reducing prefill to 1.3% of busy time and rendering execution decode-bound (Figure 6).

Deterministic context drainage eliminates hallucinated summaries. Harnesses that compact context typically invoke an LLM to generate summaries (Knutsen and Klein 2026), which costs shared GPU cycles and risks hallucinating persistent false facts into context history. Our context drain operates entirely without model inference: when context exceeds 118k tokens, it deterministically drops the oldest complete turns until history falls below 55k tokens. The system prompt remains pinned, unevicted turns are preserved verbatim, and the symbolic memory store and code graph are re-anchored as a single deterministic block.

Kernel working memory and priority scheduling. The agent maintains four level-keyed symbolic dictionaries in the Python kernel (`GOAL`, `PLAN`, `HYPOTHESES`, `FINDINGS`) that persist across context drains and level transitions. Upon level completion, a curator call consolidates confirmed mechanics into a persistent `SKILLS` repository and registers refuted rules to prevent redundant exploration. Across 25 concurrent games, an asynchronous priority scheduler allocates 10 active GPU slots based on level reward weight discounted by committed actions, multiplying by a decay factor that prevents stalled games from starving progressing ones (Appendix A).

Ablated designs and boundary conditions. Across our development cycle, several architectural alternatives proved detrimental or inert. Replacing past code with semantic stubs caused 6B models to degenerate into copying comment templates in 97.7% of calls, fine-tuning on self-generated successful traces reduced training loss while degrading interactive generalization, and spawning autonomous subagents introduced coordination overhead without performance gain. A complete analysis of these ablated designs and runtime edge cases is provided in Appendix C.

5 Results

Our empirical evaluation establishes that action budget exhaustion on ARC-AGI-3 is driven predominantly by perceptual misinduction rather than downstream planning failure. We first present the controlled experiments and out-of-distribution evaluations of the Perception Engine (Section 5.1), followed by action efficiency and error budget decompositions (Section 5.2), and conclude with hardware-matched comparative benchmarks and systems telemetry (Section 5.3).

5.1 Empirical Evaluation of the Perception Engine

Controlled perception lab on development environments. To evaluate how alternative state representations influence visual scene comprehension, we designed a controlled perception lab evaluating nine distinct presentation formats on the opening frames of `ls20` (discrete arrow-key navigation) and `r11l` (continuous coordinate click interactions). Each presentation was graded against a pre-registered 28-point rubric measuring seven essential perceptual dimensions: agent identification, goal state, mechanical links, hazards, status gauges, terrain composition, and winning plan induction. As detailed in Table 3 and Figure 7, our initial baseline (the Legacy Encoder, `arc_eye`) scored only 3 of 28 points despite consuming 3,500 text tokens per frame. Because it serialized coordinates without relational synthesis, the reasoning model bore the entire burden of deducing spatial semantics, misclassifying functional move gauges as inert screen chrome.

Representational interventions advanced comprehension across two distinct axes. First, host-side geometric computations (X3 Visual Routines and X4 Look-alike Threads) extracted spatial properties that neural vision encoders fail to resolve reliably, such as discovering midpoint tethers and identifying that the `ls20` inventory icon matches the goal glyph rotated by 90° and rescaled. Second, candidate role annotations (X7 Role Ledger) provided rapid operational context, raising the rubric score to 20/28. Synthesizing these approaches into the Fused Briefing (X8) achieved 25 of 28 points at 370 tokens per frame, representing an $8.3\times$ score improvement over the baseline alongside an 89.4% reduction in token consumption. Dumping raw 64×64 ASCII grids alongside per-cell component tables forces the tokenizer to spend $\approx 3{,}500$ tokens per frame encoding repetitive background coordinates; by abstracting connected components on the host, moving spatial layout into the visual Blueprint raster, and emitting only non-redundant geometric relations in text, the Fused Briefing compresses prompt footprint by

Presentation	ls20 /14	r11l /14	Total /28	Int. 1–5	Tokens
Legacy Encoder (baseline)	2	1	3	1	3500
X1 Exploded Inventory	2	1	3	2	180
X2 Surprise Map	3	1	4	2	270
X3 Visual Routines	7	8	15	4	240
X4 Look-alike Threads	4	2	6	3	140
X5 Foresight Strip	4	8	12	4	140
X6 Blueprint	4	4	8	3	100
X7 Role Ledger	11	9	20	5	380
X8 Fused Briefing (Ours)	**13**	**12**	**25**	**5**	**370**

Table 3 | Perception Lab evaluation across candidate presentations. Rubric scores on the opening frames of `ls20` and `r11l`, intuition rating (Int., 1–5), and text token footprint per frame. The Fused Briefing (X8) achieves 25/28 points with an 89.4% reduction in token cost relative to the Legacy Encoder baseline.

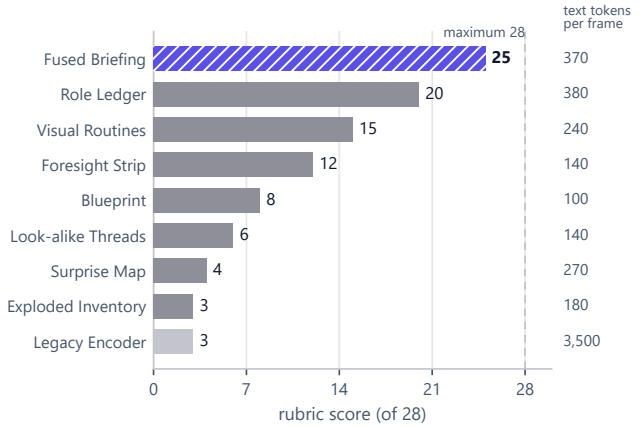


Figure 7 | Perception Lab rubric score versus token cost. The Fused Briefing establishes an empirical Pareto frontier, scoring 25/28 against 3/28 for the Legacy Encoder while reducing prompt tokens by nearly an order of magnitude.

89.4% (to 370 tokens) while increasing prefix-cache capacity by 9.5×.

Cross-distribution validation across 23 held-out games. Rubric scoring on the two development frames was conducted non-blind against a pre-registered answer key—out-of-distribution generalization is therefore verified programmatically across the 23 held-out games in Table 4. Ground-truth verification was executed programmatically against the official game engine without manual intervention. As reported in Table 4, deterministic coordinate reconstruction achieved 100% cell accuracy across all 23 games (23/23). Furthermore, computed geometric invariants (4-connected segmentation, bounding enclosures, and dihedral symmetries) proved unconditionally invariant across novel layouts.

In stark contrast, single-frame operational guesses generalized poorly out of distribution. Candidate agent identifications were accurate on only 6 of 14 keyboard environments, moves-left gauge heuristics succeeded on 10 of 20 games, and click-affordance predictions held on only 7 of 29 candidate handles. Forward predictive simulation (X5) succeeded on only a single game out of 16. These empirical findings substantiate the **Measured–Guessed Rule**: mathematical invariants hold unconditionally by construction and should be asserted as ground truth, whereas latent operational affordances cannot be reliably deduced from static visual frames and must be framed as provisional hypotheses paired with unit-action verification probes.

Check on 23 Held-Out Official Games	Empirical Generalization
Screenshot to grid, exact cells	23/23 games (100%)
Agent identity (X7), keyboard games	6/14 games (43%)
Moves-left gauge found (X7)	10/20 games (50%)
Click handles that respond (X7)	7/29 handles (24%)
Keyboard foresight fully right (X5)	1/16 games (6%)

Table 4 | Empirical validation of the Measured–Guessed Rule across 23 held-out environments. Deterministic coordinate recovery and geometric relations generalize unconditionally (100%), whereas latent affordance roles and predictive forward models from a single frame achieve between 6% and 50% accuracy, confirming that operational affordances must be framed as testable hypotheses.

5.2 Action Efficiency and Error Decomposition

Super-human action efficiency on resolved environments. On levels where the agent successfully induces environment mechanics, SARBLOH executes trajectories with high efficiency. We report two full public runs, named by the configuration that separates them. In Run A (briefing off: the briefing and map were disabled, so the run characterizes the action interface, serving loop, and scheduler), the agent cleared 143 of 183 levels across the 25 public games in 8.2 hours, recording a median action ratio of **0.77** and a single-pass mean score of 59.4%, with 66% of cleared levels matching or outperforming the human baseline (Figure 8a). This run provides the baseline for our score decomposition (Figure 8b, 40.6 total points lost from a perfect 100). Under the RHAE metric, 62% of completed levels earned the maximal score cap of 115%. In Run B (briefing on: the briefing and map enabled, Table 8), the agent recorded a median action ratio of **0.68** on cleared levels and a single-pass mean score of 63.5%. By synthesizing complete algorithmic search routines in Python and committing multi-step paths via `await play()`, the agent avoids incremental trial-and-error once the scene structure is understood.

Perceptual misinduction dominates the error budget. We decomposed the 40.6 total points lost from a perfect benchmark score using official RHAE weighting (Figure 8b). Unreached levels resulting from earlier stalls accounted for 20.1 lost points, initial uncleared levels contributed 9.2 points, and sub-optimal action overhead on cleared levels

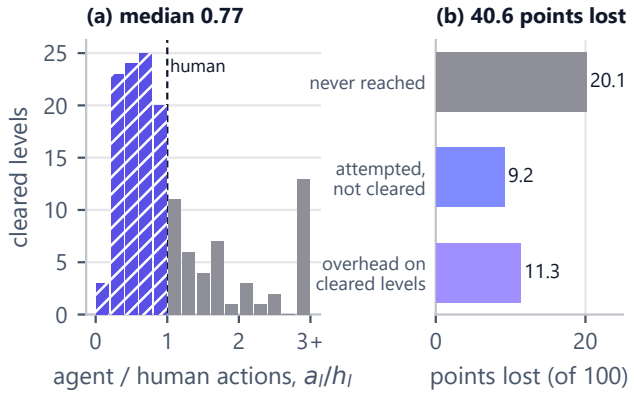


Figure 8 | Action efficiency and score error budget. (a) Distribution of agent-to-human action ratios (a_l/h_l) across 143 cleared levels (Run A, briefing off), where hatched bars indicate performance surpassing human baselines. (b) Decomposition of the 40.6 total points lost from a perfect score, illustrating that 72% of lost credit stems from uncleared levels stalled by perceptual misinduction rather than action overhead on resolved levels.

represented only 11.3 points. Consequently, 72% of all lost points (29.3 of 40.6) occurred on environments the agent failed to resolve. Trace autopsies on these stalled environments reveal that failure was driven by perceptual misinduction rather than planning depth. On stalled levels, the agent reformulated its stated goal a median of 8 times (compared to once on cleared levels) and formulated incorrect inductive beliefs, such as declaring `vc33` unsolvable after 739 actions before squandering 1,240 further moves. Misperceiving latent rules blinds the search space, reinforcing that perceptual grounding is the governing operational bottleneck.

5.3 Benchmark Trajectory and Comparative Evaluation

Official held-out Kaggle evaluation trajectory. Because the official Kaggle benchmark executes offline on undisclosed environments, it provides an uncontaminated measure of out-of-distribution reasoning (Figure 9). Initial baselines adapting Prime Agent with open weights scored between 0.00 and 0.27, repeatedly executing voluntary resets without finding a goal. Systematic architectural upgrades advanced performance to 5.14 through SGLang speculative decoding, 5.65 via unified `await play()` programmatic execution, 6.06 through deterministic context drainage, and 10.98 with persistent symbolic memory dictionaries. Integrating the full deterministic perception engine alongside refined drainage established our peak verified score of **16.53**, representing a 13.7× efficiency improvement over the first Milestone winner.

Hardware-matched comparative baseline. To isolate algorithmic contributions from compute resources, we evaluated SABRLOH directly against the first Milestone winner (the Duck harness (Tufa Labs 2026)) on identical Kaggle hardware (one NVIDIA RTX PRO 6000) and identical Qwen3.8-Flash-Next model weights. While the baseline ran four parallel passes per game (100 concurrent streams), SABRLOH executed a single scheduled pass across the 25 public games. Crucially, this is Run A, which ran with the briefing and map disabled and characterizes the action interface, serving loop, and scheduler (Appendix C.4); in contrast, the official hidden-set score of 16.53 was produced with the full perception engine active (Figure 9). As shown in Figure 10, SABRLOH consistently accelerated discovery, clearing 73 levels versus 54 at one hour, 97 versus 63 at two hours, and finishing at 143 of 183 levels in 8.2 hours versus 129 distinct levels for the baseline. This efficiency stems from our priority scheduler concentrating compute on game states with high marginal progress potential rather than partitioning resources uniformly.

Evaluation regime asymmetry and score contextualization. A critical distinction governs the comparison between offline competition submissions and frontier API benchmarks. Systems reporting near-ceiling scores on public games (Prime Agent, AVO, Arcgentica) operate across commercial API endpoints with unbounded wall-clock time, unrestricted internet access, and proprietary closed-weight backbones such as Claude Opus 5 and GPT-5.6 Sol. Furthermore, these evaluations execute on disclosed, publicly accessible environments where prompt templates and heuristic search budgets can be manually adapted without verifiable submission constraints. In contrast, SABRLOH operates within the official competition environment, where an immutable Docker container executes on a single offline GPU without external network access, constrained by a strict nine-hour wall-clock ceiling across the entire benchmark suite of undisclosed games. In this hardware-bounded regime, where every exploratory misstep incurs non-linear score degradation, achieving an official score of 16.53 represents an unprecedented result that surpasses the first Milestone winner by 13.7×, establishing that structured core-knowledge perception enables open-weight models to compete effectively without requiring unconstrained cloud resources.

Comparison with frontier model baselines and systems telemetry. On public environments, SABRLOH achieves an aggregate mean score of 72.5 across evaluation trials, outperforming proprietary frontier models evaluated via unconstrained APIs on 21 of 25 games and achieving 100% resolution on 13 games (Figure 1). Under a single-pass regime, SABRLOH recorded a mean score of 59.4 in Run A, briefing off (Table 7), and 63.5 in Run B, briefing on (Table 8), each exceeding the strongest frontier baseline (GPT-6.1 Sol at 54.8). In terms of runtime telemetry on the single Kaggle RTX PRO 6000 GPU, the append-only context preserved a 98.4% prompt prefix-cache hit rate across 1.78 B total prompt tokens, ensuring that prefill consumed only 1.3% of busy execution time compared to 50.1% for decode generation. Across 21,224 model calls, the runtime suffered zero server failures (Table 7). The comparison of this briefing-off run with the nearest briefing-on run is in Appendix C.4 (Table 8). Runtime failure modes and ablated designs are detailed in Appendix C.

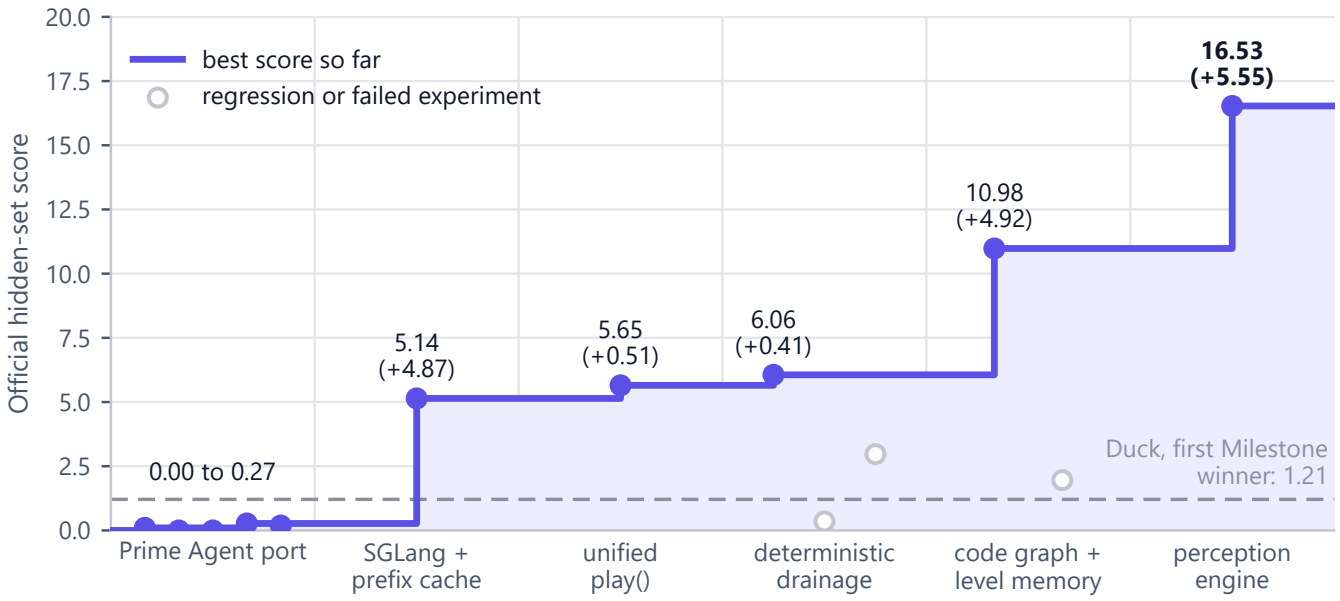


Figure 9 | Evaluation trajectory across completed submissions on the official held-out Kaggle benchmark. Submissions are ordered by intervention, not by date: each sits in the stage of the last intervention it carried, and every stage has the same width. The solid curve tracks the best score so far, hollow markers indicate regressions and failed experiments, and each label gives the stage's best score and its gain over the stages before it. The last stage's submission also changed the level memory and the context drain (Section 5.3). The official hidden-set score (16.53, produced with the full perception engine active) is evaluated on the competition-wide RHAE metric across undisclosed games, whereas Figure 1 reports per-game action efficiency percentages on public environments.

6 Related Work

Perceptual representations in interactive ARC agents. Prior ARC-AGI-3 architectures differ across planning strategies, context memory management, and external tooling, yet they share a uniform perceptual assumption: the visual grid is transmitted directly to the reasoning model as raw unparsed data, delegating spatial scene decomposition entirely to neural inference (Table 1). Specifically, the first Milestone winner (the Duck harness (Tufa Labs 2026; ARC Prize Foundation 2026b)) serializes the board as composite ASCII character matrices alongside rendered image frames. The second- and third-place Kaggle systems pass annotated screenshots to Gemma-4-31B, prompting the vision model to emit JSON action sequences directly (ARC Prize Foundation 2026b). In the frontier API regime, VISTA (Han et al. 2026) deploys a multimodal harness with lossless visual frame retention, while Arcgentica (Knutsen and Klein 2026) and Prime Agent (Karten et al. 2026) expose the environment within a persistent Python REPL, leaving the language model to write its own parsing scripts over raw coordinate arrays.

Core knowledge grounding and geometric invariants. Across all existing systems, the agent observes the world as an integer matrix, an ASCII string, or an unannotated visual raster. Consequently, fundamental spatial primitives, including 4-connected entity segmentation, topological containment, lattice-constrained reachability, and affine symmetries, are left to be reconstructed by the neural backbone from raw tokens. As our motivating ablations establish (Section 3), this unparsed representation forms the primary source of action inefficiency and hallucination. In contrast, Sarbloh computes geometric invariants deterministically on the host in $\mathcal{O}(HW)$ time, maintains persistent bipartite entity tracking across temporal steps, and explicitly decouples mathematical certainties from single-action testable affordance hypotheses.

Autonomous code generation and programmatic execution. The CodeAct paradigm (Wang et al. 2024) demonstrated that expressing agent actions through executable programming primitives outperforms specialized JSON action schemas. Prime Agent (Karten et al. 2026) adapted this to ARC-AGI-3 by allowing models to execute Python code within an interactive environment. However, earlier harnesses decoupled model reasoning into disjoint tools for code simulation and action execution, forcing expensive context-switching roundtrips. Sarbloh unifies these operations within an asynchronous execution contract (`await play()`), allowing models to hypothesize, simulate, and commit multi-step actuation trajectories within a single reasoning turn.

7 Limitations and Future Work

Limitations. Several architectural limitations remain. First, while affordance hypotheses are coupled with unit falsification tests, the current perception engine evaluates hypotheses independently at each frame without maintaining a fully unified Bayesian belief distribution across multi-step episodes. Second, although the agent plans with super-human

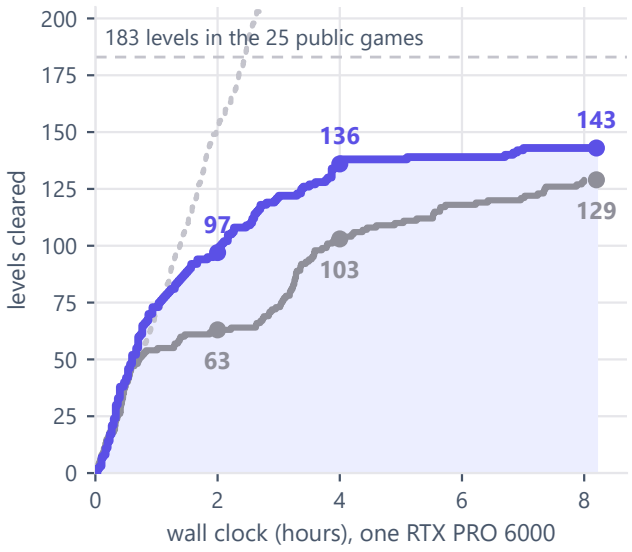


Figure 10 | Cleared levels over wall-clock execution time on 25 public environments under identical hardware. Indigo: SARBLOH (single pass, Run A, briefing off). Solid grey: distinct levels cleared by the first Milestone winner (Duck harness) across four parallel passes; dotted grey: total levels summed across passes. SARBLOH clears 143 of 183 levels in 8.2 hours, surpassing the baseline's 129 distinct levels.

efficiency once a level is understood, antecedent failure on complex perceptual transitions remains the primary source of score degradation. Finally, our current results rely on general-purpose open-weight models without task-specific policy fine-tuning.

Future work. Promising research avenues include:

1. **Closed-Loop Belief Updating**: Formulating affordance induction as a dynamic partially observable belief state, where the empirical outcomes of unit falsification probes explicitly update prior belief weights across compositional levels.
2. **Offline Trajectory Fine-Tuning**: Applying parameter-efficient fine-tuning (LoRA (Hu et al. 2021)) directly on our database of verified, high-efficiency execution trajectories, optimizing syntactic code generation and world model synthesis.
3. **Sample-Efficient Reinforcement Learning**: Investigating offline reinforcement learning algorithms, such as Group Relative Policy Optimization (GRPO (Shao et al. 2024)) and balanced policy clipping, to optimize planning depth and action parsimony within our fixed computational envelope.

8 Conclusion

Autonomous reasoning in uninstructed interactive environments is fundamentally constrained by perception rather than downstream algorithmic planning. Under the non-linear action penalties of ARC-AGI-3, agents cannot afford to compensate for impoverished representations through exploratory trial and error. As our empirical investigations demonstrate, presenting environments as unparsed coordinate matrices induces perceptual misinduction, compelling language and vision backbones to squander their action budgets attempting to reconstruct topological adjacency and basic spatial mechanics.

To resolve this limitation, we introduced SARBLOH, an autonomous agent architecture organized around the Measured–Guessed Rule. By computing deterministic geometric invariants directly on the host in linear time, the perception engine extracts lattice reachability, topological enclosures, midpoint tethers, and dihedral symmetries as mathematical certainties that generalize unconditionally across novel environments. Simultaneously, the framework treats latent operational semantics not as brittle classifications, but as provisional inductive hypotheses paired with minimal unit-action verification probes. Grounded by this perceptual foundation, the agent models environment transitions through programmatic Python execution, dispatching multi-step actuation trajectories that halt dynamically upon encountering unexpected state transitions.

Our findings indicate that advancing fluid intelligence on novel benchmarks does not strictly necessitate larger parameter scales or unconstrained inference budgets. Instead, equipping reasoning models with structured, core-knowledge perceptual abstractions bridges the divide between visual observation and symbolic deduction, enabling open-weight architectures to explore with disciplined, sample-efficient physical agency.

References

ARC Prize Foundation. 2026a. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. *arXiv preprint arXiv:2603.24621*.

ARC Prize Foundation. 2026b. ARC Prize 2026: ARC-AGI-3 Milestone #1 Results. https://arcprize.org/blog/arc-prize-2026-milestone-1.

Cheng, W.; Zhang, W.; Shen, H.; Cai, Y.; He, X.; et al. 2023. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. *arXiv preprint arXiv:2309.05516*.

Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.

Han, Q.; Hu, K.; Qiu, L.; Wu, C.; and He, K. 2026. VISTA: A Visual Harness for Reasoning in an Interactive World. https://vista-research.github.io/.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Karten, S.; Zhang, A. L.; Thomas, K.; Müller, S.; Bakouch, E.; et al. 2026. Prime Agent: A Self-Improving RLM Harness. *arXiv preprint arXiv:2608.23552*.

Knutsen, S.; and Klein, V. 2026. Arcgentica: ARC-AGI-3 Agent Harness Built on the Agentica SDK. https://github.com/symbolica-ai/ARC-AGI-3-Agents.

Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *arXiv preprint arXiv:2401.15077*.

Marr, D. 1982. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman.

Rahmanzadehgervi, P.; Bolton, L.; Taesiri, M. R.; and Nguyen, A. T. 2024. Vision Language Models Are Blind: Failing to Translate Detailed Visual Features into Words. *arXiv preprint arXiv:2407.06581*.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Spelke, E. S.; and Kinzler, K. D. 2007. Core Knowledge. *Developmental Science*, 10(1): 89–96.

Tufa Labs. 2026. The Duck: Tufa Labs' ARC-AGI-3 Harness, Milestone #1 Winner. Kaggle notebook, https://www.kaggle.com/code/jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner.

Ullman, S. 1984. Visual Routines. *Cognition*, 18(1-3): 97–159.

Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; et al. 2024. Executable Code Actions Elicit Better LLM Agents. *arXiv preprint arXiv:2402.01030*.

Yang, S.; Kautz, J.; and Hatamizadeh, A. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.

Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; et al. 2023. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104*.

A Notation and Priority Scheduler

Table 5 collects the symbols used in the main text and in this appendix.

Symbol	Meaning
Environment and score (Section 2)	
g_t	grid observed at step t, in $\{0..15\}^{H\times W}$
$H, W \le 64$	grid height and width
$\mathcal{A}$	action set, Equation (1)
n; l; k	levels in a game; level index; levels cleared
h_l; a_l	human baseline actions; agent actions on level l
S_l	level score, Equation (2)
E	game score, Equation (3)
Perception engine (Algorithm 1)	
$\mathcal{T}$	terrain colours, each covering $>8\%$ of HW
L	layout: the things of a frame (objects, corridors, gauges, panels)
Λ; Λ'	detected tile lattice; lattice of the previous frame
$D_4 \times \mathbb{Z}^+$	the eight rotations and mirrorings of the square, with integer scaling
F	measured facts (reach, enclosure, tethers, symmetries)
e; $\boldsymbol{\sigma}_e$	a thing; its role scores
$\hat{\rho}_e$; p_e	guessed role of e; confidence of that guess
Scheduler (Equation (4))	
π	priority of a game for a GPU slot
g_{now}; g_{tail}	value of the current level; of the levels after it
m; a	reply tokens; actions on the current level
h; m_0; a_0	25 actions; 40k tokens; 115 actions
τ_{tail}; ϕ	tail weight by levels left; end-of-run fade

Table 5 | Notation.

Priority scheduler. Figure 11 shows the whole system. Twenty-five games share ten GPU slots, and the scheduler gives a free slot to the waiting game with the highest expected score gain. For a game that has cleared k of its n levels and has spent m reply tokens and a actions on level $k+1$ without clearing it, the priority is

$$\pi = \big(g_{\text{now}} + g_{\text{tail}}\big) \cdot \underbrace{2^{-\left((m/m_0)^2+(a/a_0)^2\right)}}_{\text{hope}}, \tag{4}$$

$$g_{\text{now}} = \frac{k+1}{n(n+1)/2}\left(\frac{h}{h+a}\right)^2, \quad g_{\text{tail}} = \frac{\tau_{\text{tail}}(n-k-1)}{55}\,\phi.$$

g_{now} is the level's weight in Equation (3), discounted quadratically by the actions already spent on it; g_{tail} is the value of the later levels that clearing it unlocks. We set $h = 25$ actions (a typical human level), $m_0 = 40$k tokens and $a_0 = 115$ actions. $\tau_{\text{tail}}(j)$ is 8, 7, 5 or 0 when $j \ge 3$, 2, 1 or 0 levels remain after the current one, on the 55-weight scale of a 10-level game, so short games do not starve long ones. $\phi \in [0, 1]$ falls linearly to zero over the last 20% of the run, when there is no time left to reach later levels. A game that has never held a slot goes first, and a game keeps its slot for 20 requests before the scheduler looks again. The constants were set by hand, without a search. Equation (4) is the function `priority` in `Sarbloh/harness/scheduler.py`.

A.1 Hyperparameters

Table 6 lists the settings of the two full public-game runs of Appendix C.4, read from the configuration file each run saved with its results and from its server log. The two runs differ only where a cell gives two values.

Parameter	Value
Games / GPU slots	25 / 10
Scheduler quantum	20 model calls
Action cap per game	2,000 actions
Actions per `play()` path	at most 100
Reply cap per request / requests per turn	12k tokens / 12
Thinking without a move before the turn yields	16,384 tokens
Context drain trigger / target	118k / 55k (Run A), 70k (Run B)
Context window	131,072 tokens
Map scale	10 pixels per grid cell
Briefing lines / short-view text cap	20 lines / 600 tokens
Memory caps: goal / plan / hypotheses / findings	150 / 200 / 400 / 300 tokens
Temperature / top-p / top-k	0.7 / 0.95 / 20
Serving	SGLang, W4A16 (AutoRound), FP8 KV cache
Speculative decoding	EAGLE with a multi-token-prediction drafter, 3 steps, 4 draft tokens

Table 6 | Settings of Run A (briefing off) and Run B (briefing on).

B Perception Lab Protocols and Full Briefings

This appendix gives the protocol of the perception lab, the eight presentations it compared, and the full briefings for the two lab frames. The results are in Section 5.1 (Table 3, Figure 7 and Table 4).

B.1 Setup and Answer Key

Shared front end. All experimental arms (X1–X8) share an identical front end to isolate representational formatting from extraction noise. A raw frame is transformed into an exact 64×64 categorical color grid by fitting cell pitch and origin from chromatic boundary gradients. The grid is segmented into background terrain (modal color coverage exceeding 8%), four-connected monochromatic entities categorized by morphology (compact objects, icons, dotted structures, thin connectors, or bars), bounding panels, and peripheral heads-up display (HUD) elements within three cells of the screen boundary.

Baseline encoder. The baseline is our legacy text-only encoder (`arc_eye`). It extracts the fitted lattice, assigns a fixed palette symbol per cell, performs connected component segmentation with hole detection, computes dihedral shape hashes, and identifies bounding boxes alongside inter-frame

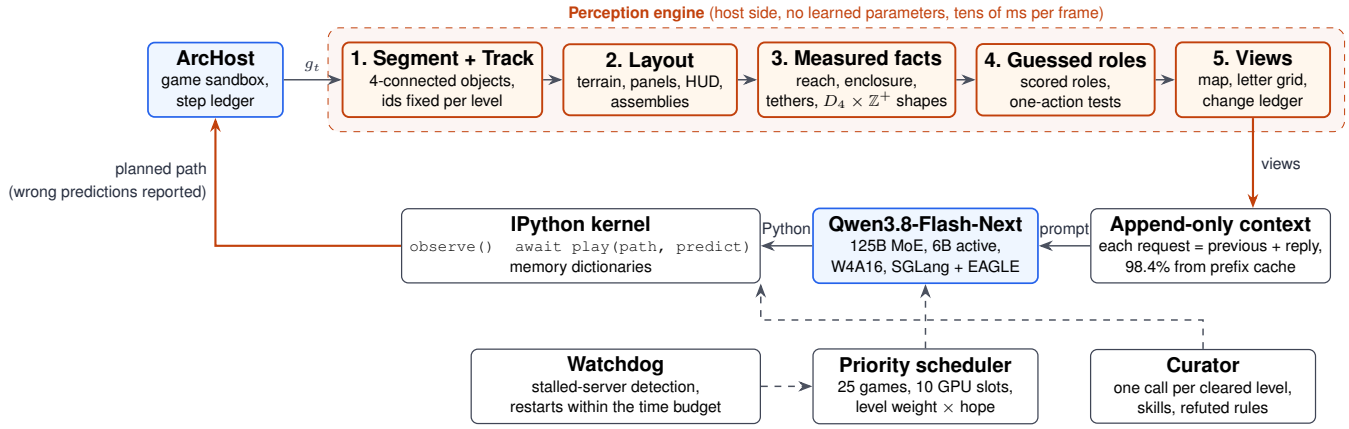


Figure 11 | System topology of SARBLOH. The host-side perception engine extracts invariant relations and affordance hypotheses from each frame, passing views to an append-only context. The reasoning model generates Python search routines in a persistent kernel, committing multi-step paths via `await play()`. Twenty-five games share ten GPU execution slots under the utility-driven priority scheduler.

coordinate differences. It emits an exhaustive text dump requiring approximately 3,500 tokens per frame without visual images.

Answer key and scoring. The evaluation suite comprises the opening levels of `ls20` (arrow-key lattice navigation) and `r11l` (continuous coordinate click interactions). Before any arm was run, an answer key of seven items was written for each frame:

K1 the controllable entity;

K2 the goal;

K3 the link or carried object;

K4 the modifier or hazard;

K5 the status gauges;

K6 the winning plan;

K7 the terrain boundaries.

Each item scored 2 for a clear and correct description, 1 for a partial one, 0 if it was missing and -1 if it was misleading, so each game has a maximum of 14 points and the two together 28. An intuition rating from 1 to 5 and the prompt token cost were recorded alongside.

B.2 Candidate Presentations

X1 Exploded Inventory. Entities are de-duplicated by exact shape grids, magnified to a standard display scale, arranged spatially with letter tags, and annotated with multiplicity counts ($\times N$). A miniature thumbnail outlines original entity positions. Thin connectors are cataloged by endpoint coordinates.

X2 Surprise Map. Calculates cell-level information surprise as $-\log_2 p$ based on 3×3 neighborhood pattern distributions. Periodic floor tiles and boundary walls score near zero bits. Entities are ranked by total bits, preserving full saturation only on high-surprise components while fading predictable background textures.

X3 Visual Routines. Executes serial geometric routines based on Ullman (1984). **Reach** performs breadth-first search across detected lattice steps, reporting path lengths from the likely agent to all visible targets. **Fit** correlates hollow enclosing boundaries with objects of identical dimensions. **Trace** tracks single-pixel connectors to their endpoints, identifying entities suspended at the midpoint of dual anchors. **Align** catalogs entities sharing identical row or column spans.

X4 Look-alike Threads. Compares binary entity shapes under all eight dihedral symmetries of the square alongside integer scaling factors ($D_4 \times \mathbb{Z}^+$). Matched pairs are classified as EXACT (identical copies), TURNED (matched under rotation or reflection), NEAR (differing by at most three cells), or STATE (identical geometry with modified color palettes). Visual threads connect matched entities across the frame.

X5 Foresight Strip. Projects prospective states for legal actions using hardcoded physical priors: agent sprites translate by their bounding dimensions until striking walls, edge gauges decrement by unit length per action, and midpoint objects track anchor displacements.

X6 Blueprint. Re-renders the scene as an architectural schematic following Marr (1982): terrain rendered in soft tones, structural boundaries hatched, tile lattices faintly gridded, and HUD elements isolated into a dedicated side panel.

X7 Role Ledger. Scores non-terrain entities against eight standard gameplay roles (YOU, GOAL, MOVES LEFT, LIVES, INVENTORY, MODIFIER, HANDLE, CARRIED THING) using evidence heuristics, printing each top candidate alongside an empirical unit-action verification test.

X8 Fused Briefing. Synthesizes the architectural blueprint canvas of X6, the relational threads and midpoint indicators of X3–X4, and a two-tiered text briefing explicitly separating MEASURED geometric certainties from

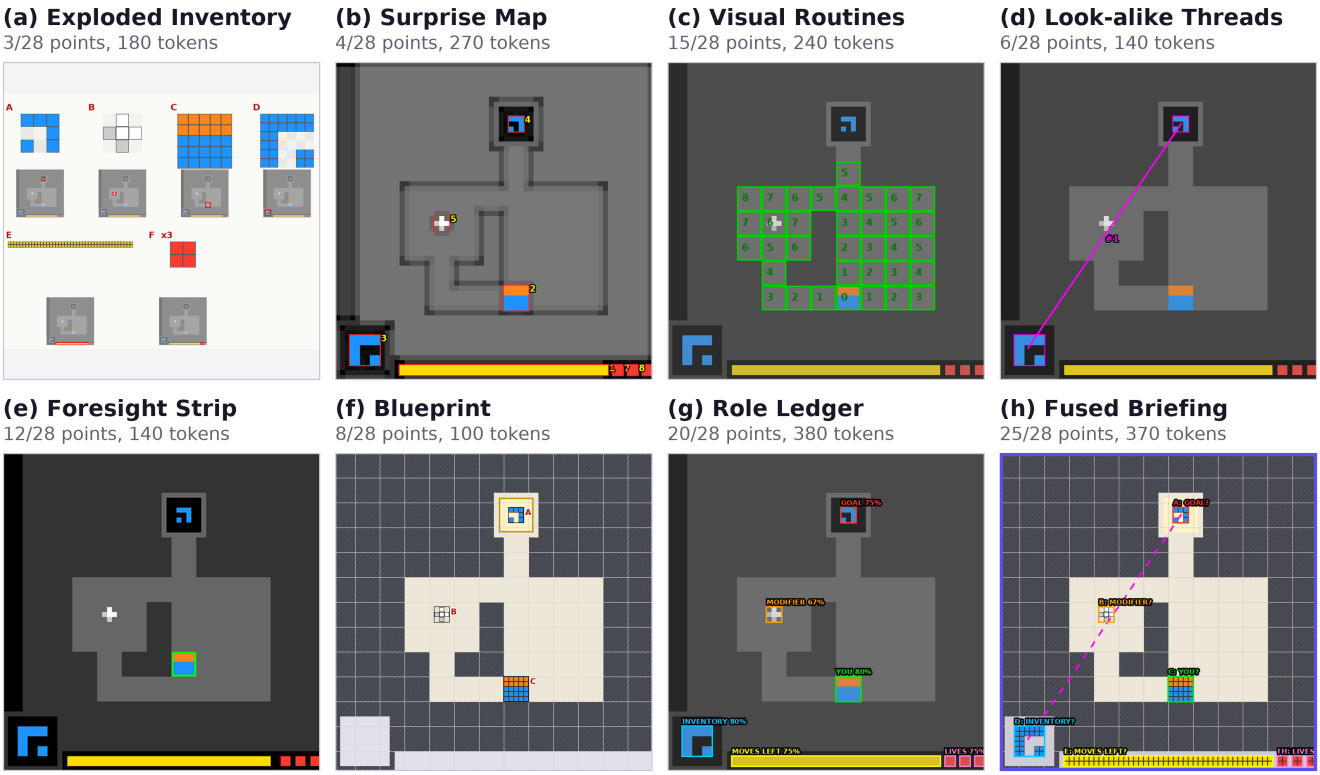


Figure 12 | Visual presentations evaluated in the Perception Lab on `ls20`. (a) Exploded Inventory separates unique entity morphologies. (b) Surprise Map highlights unpredicted local textures. (c) Visual Routines computes lattice BFS step counts to candidate targets. (d) Look-alike Threads connects shapes matching under rotation, reflection, or integer scaling. (e) Foresight Strip simulates counterfactual forward states. (f) Blueprint redraws terrain and lattices as a clean architectural schematic. (g) Role Ledger annotates candidate affordance hypotheses. (h) Fused Briefing combines the blueprint map with measured invariants and testable role hypotheses.

GUESSED affordance hypotheses paired with unit-action verification tests.

Figure 12 provides a visual comparison of each candidate presentation evaluated on `ls20`.

B.3 Full Structural Briefings

The two boxes below give the briefing text the perception engine wrote, word for word, for the opening frames of `ls20` (Figure 5) and `r11l`. The object ids are the same as those on the map and in the kernel's object table (`observe().objects`).

`ls20`, opening frame (arrow keys, tile grid)

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box
    that is NOT plain floor; the floor reaches the
    box's bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
- obj 4 (blue 6-cell shape) is directly above obj 10
    (blue/orange square) (shared columns 35-37)
- obj 12 (blue irregular shape (24 cells)) (drawn 2x
    bigger), mirrored left-right OR turned 90 deg
    clockwise, becomes exactly obj 4 (blue 6-cell
    shape)
- 3 identical red square (2x2): obj 16, 17, 18 -> a
    count of something (lives, items, tiles)
- obj 6 (white/light grey plus) has a symmetric shape
    but lopsided colours (marks a direction/state)
- obj 10 (blue/orange square) has a symmetric shape
    but lopsided colours (marks a direction/state)
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test:
    press one arrow key: it should move one tile and
    the gauge should drop
- obj 12 (blue irregular shape (24 cells)) =
    INVENTORY (80%) - test: watch it after touching
    the modifier
- obj 4 (blue 6-cell shape) = GOAL (75%) - test: get
    the right thing into it and see if the level
    ends
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take
    any action and check it shrinks by one unit
- obj 16, 17, 18 (red square) = LIVES (75%) - test:
    watch it when the gauge runs out or you fail
```

```
- obj 6 (white/light grey plus) = MODIFIER (67%) -
    test: step onto it, then check whether the
    inventory icon changed
```

r111, opening frame (clicks, no tile grid)

```
BRIEFING (clicks)
MEASURED (certain):
- no tile grid -> free movement. Open space = black
    (81% of screen).
- all 4 main objects share one open space; straight
    paths between them are ALL CLEAR of rock
- the grey areas are not open floor (likely walls)
- obj 4 (purple dotted ring) encloses an EMPTY 5x5
    space -- exactly the size of: obj 16 (white/
    purple diamond), obj 25 (purple/magenta disc),
    obj 36 (dark grey/purple diamond)
- a light grey line connects obj 16 (white/purple
    diamond) to obj 25 (purple/magenta disc)
- a light grey line connects obj 25 (purple/magenta
    disc) to obj 36 (dark grey/purple diamond)
- obj 25 (purple/magenta disc) sits at the MIDPOINT
    of obj 16 (white/purple diamond) and obj 36 (
    dark grey/purple diamond) (error 0.5 cells) ->
    it is held between them; moving either end
    should move it to the new midpoint
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) have the same shape in different
    colours (white/purple vs dark grey/purple)
GUESSES (check with one action):
- obj 4 (purple dotted ring) = GOAL (80%) - test: get
     the right thing into it and see if the level
    ends
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) = HANDLE (click) (80%) - test: click an
     empty spot once: the selected handle should
    jump there
- obj 1 (white bar) = MOVES LEFT (75%) - test: take
    any action and check it shrinks by one unit
- obj 25 (purple/magenta disc) = CARRIED THING (75%)
    - test: move one handle: it should slide to the
    new midpoint
```

B.4 The Perception Pipeline on Four More Games

Figures 13–16 repeat Figure 5 for the opening frames of `ka59`, `m0r0`, `r111` and `sb26`. They were made in the same way: `make_perception_figure.py` runs the agent's perception code on the first frame of each local game, and panels (b) and (c) are drawn from its object table and briefing. Each line in (c) stands for one MEASURED line of the briefing in (e). Where an object is made of one-cell parts (the dotted ring and the connecting lines in `r111`) or repeats (the four grey squares in `sb26`), the parts are outlined and the group gets one tag. The four games test different parts of the engine. `ka59` has a tile grid, and two rings each enclose an empty 3×3 that fits a movable square. `m0r0` is sparse: four objects, two measured facts and no guesses, so the engine does not invent roles it has no evidence for. `r111` is driven by clicks, and its disc sits at the midpoint of two connected handles. `sb26` is the densest: its briefing reaches the line limit, and the remaining eight facts are left to `observe().briefing` in the kernel, as the last MEASURED line says.

C Negative Results, Ablations, and Runtime Boundary Conditions

This appendix covers the designs that failed or made the agent unstable in live runs, the runtime failure modes, and the two full runs nearest to a perception ablation.

C.1 Perceptual Edge Cases and Line Budget Quotas

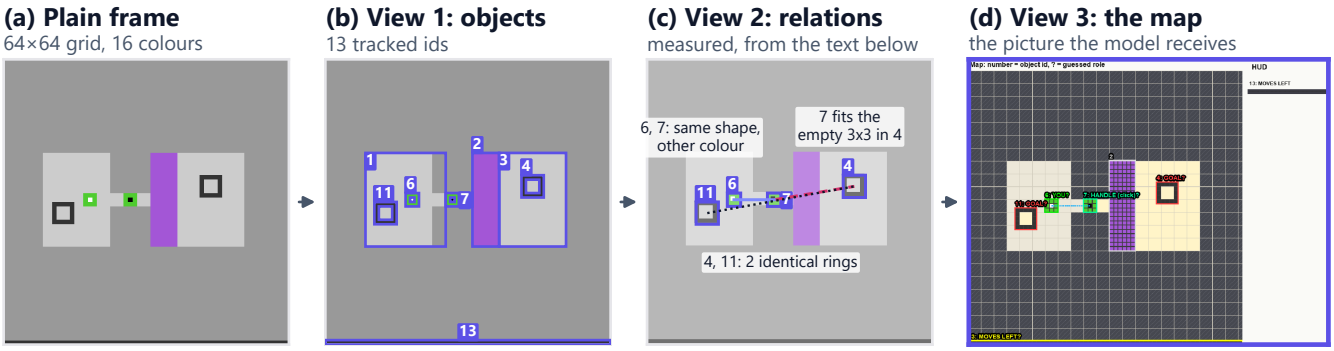
Lattice detection instability from peripheral HUD elements. In early iterations, the host-side lattice detection module fitted tile dimensions over all non-terrain pixels. In `ls20`, when the player moved, an off-grid gauge within the status box introduced irregular coordinate intervals, causing the fitted 5×5 grid to collapse. Similarly, in `ka59`, touching a secondary sprite altered the detected lattice periodicity. Each collapse temporarily deleted all reachability invariants from context, causing the agent to vacillate between conflicting spatial hypotheses. We resolved this by explicitly excluding peripheral HUD regions from the lattice fitting routine and caching the initial valid lattice configuration for the remainder of the level.

Combinatorial relation crowding. In scenes with many objects, pairwise relations grow quadratically. The first frame of `tr87` yields 60 alignment pairs, 7 enclosures and 1 symmetry, and `bp35` yields 792 alignment pairs. Under one global line cap the alignments fill the briefing and push out the symmetry and the role guesses (Figure 17a). The deployed policy shares the lines between fact kinds, keeps up to a third of them for guesses and reports the rest as a count (Figure 17b).

C.2 Rejected Architectural Designs

Sparse delta state encoding. Transmitting only differential coordinate changes between frames substantially reduces prompt token consumption. However, in ARC-AGI-3 environments where local actions trigger distant causal effects (e.g., activating a floor plate updating an icon across the display), sparse updates conceal non-local covariance. In development traces, models receiving delta updates consistently concluded that actions were non-functional when localized pixels remained static, leading to catastrophic action loops. Full-frame visual representations remain essential for maintaining causal coherence.

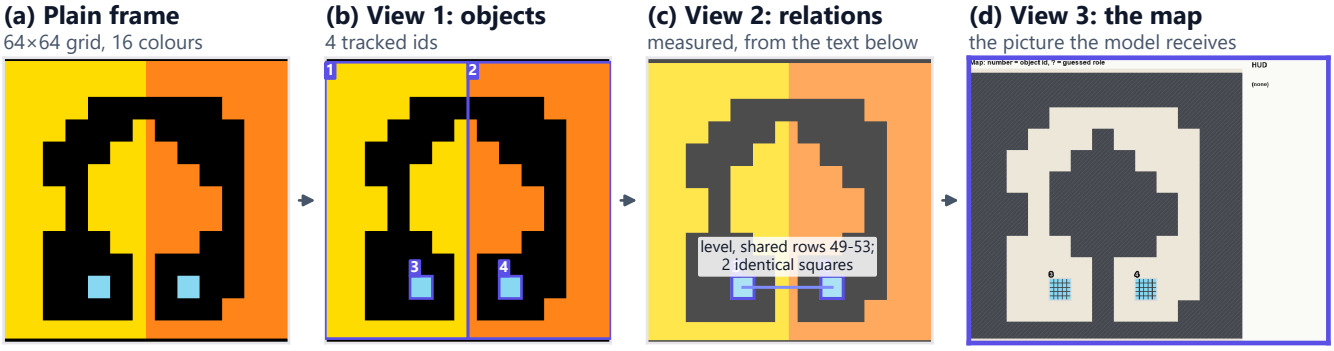
Disjoint tool spaces for reasoning and actuation. Providing separate tool interfaces for code execution and environment stepping caused models to fragment reasoning trajectories. The agent formulated world models in code, exited the REPL to dispatch individual actions, and re-entered code execution to reconstruct state variables, multiplying inference latency. Merging action commitment directly into the Python kernel via `await play()` was the single intervention responsible for advancing the hidden-set score from 5.14 to 5.65.



(e) The briefing: sent as text in the same message as (d), word for word

```
BRIEFING (arrow keys and clicks)
MEASURED (certain):
- Floor is a 3x3 tile grid.
- obj 2 (purple rectangle): reachable in 5 steps
- obj 4 (charcoal ring): reachable in 8 steps
- obj 7 (green/black square): reachable in 3 steps
- obj 11 (charcoal ring): reachable in 1 steps
- obj 4 (charcoal ring) encloses an EMPTY 3x3 space -- exactly the size of: obj 6 (green/white square), obj 7 (green/black square)
- obj 11 (charcoal ring) encloses an EMPTY 3x3 space -- exactly the size of: obj 6 (green/white square), obj 7 (green/black square)
- obj 2 (purple rectangle) is level with and left of obj 4 (charcoal ring) (shared rows 26-30)
- obj 2 (purple rectangle) is level with and right of obj 6 (green/white square) (shared rows 30-32)
- obj 2 (purple rectangle) is level with and right of obj 11 (charcoal ring) (shared rows 32-36)
- obj 6 (green/white square) is level with and left of obj 7 (green/black square) (shared rows 30-32)
- obj 2 (purple rectangle) is level with and right of obj 7 (green/black square) (shared rows 30-32)
- obj 6, 7 (green/white square; green/black square) have the same shape in different colours (green/white vs green/black)
- 2 identical charcoal ring (5x5): obj 4, 11 -> a count of something (lives, items, tiles)
GUESSES (check with one action):
- obj 4, 11 (charcoal ring) = GOAL (75-80%) - test: get the right thing into it and see if the level ends
- obj 13 (charcoal bar) = MOVES LEFT (75%) - test: take any action and check it shrinks by one unit
- obj 7 (green/black square) = HANDLE (click) (67%) - test: click an empty spot once: the selected handle should jump there
- obj 6 (green/white square) = YOU (57%) - test: press one arrow key: it should move one tile and the gauge should drop
```

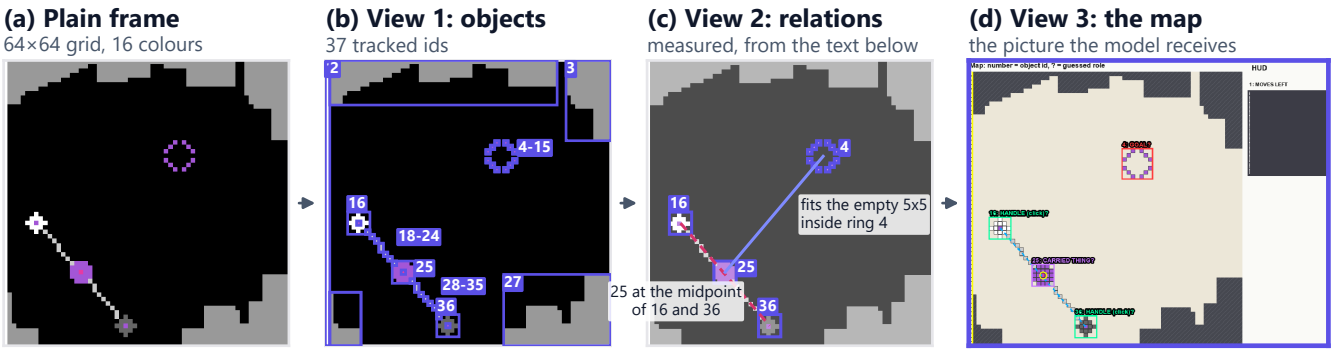
Figure 13 | The perception pipeline on the opening frame of `ka59` (arrow keys and clicks, 3×3 tile grid). (c) Rings 4 and 11 are identical; ring 4 encloses an empty 3×3 that exactly fits square 7 (and square 6); squares 6 and 7 have the same shape in different colours. Panels as in Figure 5; the model receives (d) and (e).



(e) The briefing: sent as text in the same message as (d), word for word

```
BRIEFING (arrow keys and clicks)
MEASURED (certain):
- obj 3 (sky blue square) is level with and left of obj 4 (sky blue square) (shared rows 49-53)
- 2 identical sky blue square (5x5): obj 3, 4 -> a count of something (lives, items, tiles)
GUESSES (check with one action):
- (none)
```

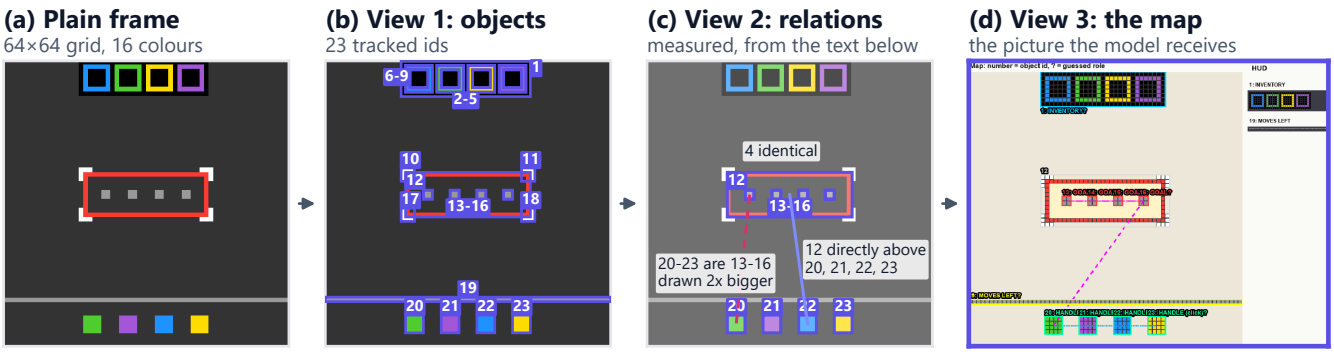
Figure 14 | The perception pipeline on the opening frame of `m0r0` (arrow keys and clicks). The engine finds four objects and measures two facts: squares 3 and 4 are identical and share rows 49–53. It makes no guesses. Panels as in Figure 5.



(e) The briefing: sent as text in the same message as (d), word for word

```
BRIEFING (clicks)
MEASURED (certain):
- no tile grid -> free movement. Open space = black (81% of screen).
- all 4 main objects share one open space; straight paths between them are ALL CLEAR of rock
- the grey areas are not open floor (likely walls)
- obj 4 (purple dotted ring) encloses an EMPTY 5x5 space -- exactly the size of: obj 16 (white/purple diamond), obj 25 (purple/magenta
  disc), obj 36 (dark grey/purple diamond)
- a light grey line connects obj 16 (white/purple diamond) to obj 25 (purple/magenta disc)
- a light grey line connects obj 25 (purple/magenta disc) to obj 36 (dark grey/purple diamond)
- obj 25 (purple/magenta disc) sits at the MIDPOINT of obj 16 (white/purple diamond) and obj 36 (dark grey/purple diamond) (error 0.5
  cells) -> it is held between them; moving either end should move it to the new midpoint
- obj 16, 36 (white/purple diamond; dark grey/purple diamond) have the same shape in different colours (white/purple vs dark
  grey/purple)
GUESSES (check with one action):
- obj 4 (purple dotted ring) = GOAL (80%) - test: get the right thing into it and see if the level ends
- obj 16, 36 (white/purple diamond; dark grey/purple diamond) = HANDLE (click) (80%) - test: click an empty spot once: the selected
  handle should jump there
- obj 1 (white bar) = MOVES LEFT (75%) - test: take any action and check it shrinks by one unit
- obj 25 (purple/magenta disc) = CARRIED THING (75%) - test: move one handle: it should slide to the new midpoint
```

Figure 15 | The perception pipeline on the opening frame of `r11l` (clicks, no tile grid). (b) The dotted ring (4–15) and the two connecting lines (18–24, 28–35) are made of one-cell parts and are tagged once each. (c) Disc 25 sits at the midpoint of handles 16 and 36, and fits the empty 5×5 inside ring 4. Panels as in Figure 5.



(e) The briefing: sent as text in the same message as (d), word for word

```
BRIEFING (clicks)
MEASURED (certain):
- no tile grid -> free movement. Open space = charcoal (89% of screen).
- the 5 main objects are in 2 separate open areas; straight paths between them are clear except: obj 12 (red/white ring) -> obj 20
  (green square) (5 rock cells); obj 12 (red/white ring) -> obj 21 (purple square) (4 rock cells); obj 12 (red/white ring) -> obj 22
  (blue square) (4 rock cells); obj 12 (red/white ring) -> obj 23 (yellow square) (5 rock cells)
- obj 12 (red/white ring) encloses a 10x28 space
- obj 12 (red/white ring) is directly above obj 20 (green square) (shared columns 18-21)
- obj 12 (red/white ring) is directly above obj 21 (purple square) (shared columns 26-29)
- obj 12 (red/white ring) is directly above obj 22 (blue square) (shared columns 34-37)
- obj 12 (red/white ring) is directly above obj 23 (yellow square) (shared columns 42-45)
- obj 13 (grey square) is level with and left of obj 14 (grey square) (shared rows 29-30)
- obj 13 (grey square) is level with and left of obj 15 (grey square) (shared rows 29-30)
- obj 13 (grey square) is level with and left of obj 16 (grey square) (shared rows 29-30)
- obj 14 (grey square) is level with and left of obj 15 (grey square) (shared rows 29-30)
- obj 14 (grey square) is level with and left of obj 16 (grey square) (shared rows 29-30)
- obj 14 (grey square) is directly above obj 21 (purple square) (shared columns 28-29)
- obj 20, 21, 22, 23 (green square; purple square; blue square; yellow square) are obj 13, 14, 15, 16 (grey square) drawn 2x bigger
- obj 20, 21, 22, 23 (green square; purple square; blue square; yellow square) have the same shape in different colours (green vs purple
  vs blue vs yellow)
- 4 identical grey square (2x2): obj 13, 14, 15, 16 -> a count of something (lives, items, tiles)
- (+8 more facts: `observe().briefing` in ipython)
GUESSES (check with one action):
- obj 20, 21, 22, 23 (green square; purple square; blue square; yellow square) = HANDLE (click) (86%) - test: click an empty spot once:
  the selected handle should jump there
- obj 19 (grey bar) = MOVES LEFT (75%) - test: take any action and check it shrinks by one unit
- obj 1 (black/blue rectangle) = INVENTORY (67%) - test: watch it after touching the modifier
- obj 13, 14, 15, 16 (grey square) = GOAL (50%) - test: get the right thing into it and see if the level ends
```

Figure 16 | The perception pipeline on the opening frame of `sb26` (clicks only). (c) Squares 20–23 are the grey squares 13–16 drawn 2× bigger, and ring 12 sits directly above each of them. (e) The briefing reaches its line limit; the last MEASURED line says eight more facts can be read from `observe().briefing`. Panels as in Figure 5.

(a) one cap of 30 lines

23 alignment, 7 enclosure, 0 look-alike, 0 guess lines

```
BRIEFING (arrow keys)
obj 6 encloses empty 3x3 space
obj 8 encloses empty 3x3 space
obj 18 encloses empty 3x3 space
obj 20 encloses empty 3x3 space
obj 31 encloses empty 3x3 space
obj 33 encloses empty 3x3 space
obj 51 encloses empty 3x31 space
obj 5 left of obj 6 r5-9
obj 5 left of obj 7 r5-9
obj 5 left of obj 8 r5-9
obj 5 above obj 17 c13-17
obj 5 above obj 30 c13-17
obj 5 above obj 44 c15-17
obj 5 above obj 47 c15-17
obj 5 above obj 51 c15-17
obj 5 above obj 62 c15-17
obj 6 left of obj 7 r5-9
obj 6 left of obj 8 r5-9
obj 6 above obj 18 c23-27
obj 6 above obj 31 c23-27
obj 6 above obj 44 c23-27
obj 6 above obj 51 c23-27
obj 7 left of obj 8 r5-9
obj 7 above obj 19 c36-40
obj 7 above obj 32 c36-40
obj 7 above obj 44 c36-40
obj 7 above obj 51 c36-40
obj 8 above obj 20 c46-50
obj 8 above obj 33 c46-50
obj 17 left of obj 18 r14-18
```

(b) lines shared by kind

20 alignment, 7 enclosure, 1 look-alike, 2 guess lines

```
BRIEFING (arrow keys)
MEASURED (certain):
obj 6 encloses empty 3x3 space
obj 8 encloses empty 3x3 space
obj 18 encloses empty 3x3 space
obj 20 encloses empty 3x3 space
obj 31 encloses empty 3x3 space
obj 33 encloses empty 3x3 space
obj 51 encloses empty 3x31 space
obj 5 left of obj 6 r5-9
obj 5 left of obj 7 r5-9
obj 5 left of obj 8 r5-9
obj 5 above obj 17 c13-17
obj 5 above obj 30 c13-17
obj 5 above obj 44 c15-17
obj 5 above obj 47 c15-17
obj 5 above obj 51 c15-17
obj 5 above obj 62 c15-17
obj 6 left of obj 7 r5-9
obj 6 left of obj 8 r5-9
obj 6 above obj 18 c23-27
obj 6 above obj 31 c23-27
obj 6 above obj 44 c23-27
obj 6 above obj 51 c23-27
obj 7 left of obj 8 r5-9
obj 7 above obj 19 c36-40
obj 7 above obj 32 c36-40
obj 7 above obj 44 c36-40
obj 7 above obj 51 c36-40
obj 47 rotated = obj 62
 (+40 more facts in observe())
GUESSES (check with one action):
obj 63 = MOVES LEFT (75%), test: take any action and check i..
obj 5, 6, 7, 8, 17, 18, 19, 20, 30, 31, 32, 33, 44, 51 (blac..
```

Figure 17 | The briefing line budget on the first frame of `tr87`, both boxes built by the agent's own `Briefing` code (object descriptions shortened; `r`/`c` = shared rows/columns). The mark at the left of each line gives its kind: enclosure, alignment, look-alike, guess. (a) One cap of 30 lines over all facts in the engine's order: 23 of the 30 lines are alignments, and the look-alike pair and both guesses are cut. (b) The policy the agent runs, which shares the lines between fact kinds and keeps up to a third for guesses: it shows the look-alike pair and the two guesses and counts the 40 facts left out ("`+40 more facts`").

Semantic history comment stubbing. To conserve context window capacity, we tested replacing executed historical code cells with brief descriptive comments. In the two test runs that had stubbing switched on, reasoning backbones degenerated into copying comment strings rather than generating executable Python code (occurring in 97.7% of turns and clearing 0 of 97 levels). Smaller open-weight models heavily imitate local context formatting; historical code must remain intact as functional syntax.

Autonomous subagent hierarchies. Providing the model with programmatic subagent delegation primitives (following the Prime Agent specification) yielded zero productive invocations across all benchmark runs. Open-weight 6B-active models failed to effectively coordinate subordinate contexts, adding prompt complexity without empirical utility.

Self-imitation fine-tuning on cleared trajectories. We fine-tuned Qwen3.8-Flash-Next on successful execution traces using low-rank adaptation (LoRA). While training cross-entropy loss decreased from 11% to 6%, out-of-distribution benchmark performance degraded significantly. The fine-tuned weights exhibited rigid memorization of specific coordinate paths rather than generalized spatial reasoning, consuming over 80% of our training compute budget without positive yield.

C.3 Interactive Runtime Failure Modes

Degenerate holding loops at the action cap. In Run A (briefing off), four games (`lf52`, `s5i5`, `vc33`, `re86`) stopped 2 to 15 actions short of their 2,000-action cap. The harness kept asking them to continue, and the model answered "`Holding...`" about 14,000 times, which used 1.24 B of the run's 1.78 B prompt tokens. The stalls began after these games had cleared every level they would clear, so they cost wall clock, not score. The harness has a per-game turn limit (400 by default), but both full runs set it to 10^6, so no run reported here was protected by it; whether the limit removes the stall without costing levels is untested.

C.4 Runtime Telemetry and the Nearest Briefing Ablation

The run was decode-bound. Table 7 gives the telemetry of Run A, briefing off (8.2 h, 25 games, one RTX PRO 6000), from the run's speed report and server log. Because 98.4% of the prompt tokens came from the prefix cache, prefill took 1.3% of busy time and decoding 50.1%. The server answered 21,224 model calls without a failure.

The 143-level run sent no briefing. Run A was made while the briefing and map were switched off: the model saw the plain 640×640 frame and the letter grid. Its numbers describe the action interface, the serving loop and the scheduler, not the perception engine.

Run A: 25 games, one GPU, 8.2 h	Value
Prompt tokens served from the prefix cache	**98.4%** of 1.78 B
Decode speed per request	76 tok/s
Busy time: decode / prefill	50.1% / 1.3%
Model calls / server failures	21,224 / 0

Table 7 | Systems telemetry of Run A (briefing off).

The closest briefing-on run cleared levels more slowly but more efficiently. Run B had the briefing and map on, together with new guardrails and a stuck-level review, and ran for 5.4 h (Table 8). It cleared fewer levels in the first two hours (51 against 74 at 1 h) and drew level at about 4 h, while its median action ratio on cleared levels fell from 0.77 to 0.68 and its mean public score rose from 59.4 to 63.5. The runs differ in more than the briefing and there is one run per arm, so we report this as consistent with fewer wasted actions once a level is understood, not as a measured effect of perception.

25 public games, one GPU Logged (2026)	Run A 23 Oct	Run B 27 Oct
Configuration		
Briefing and map	off	on
Extra guardrails, stuck review	–	✓
Context drain target	55k	70k
Wall clock	8.2 h	5.4 h
Outcome		
Levels cleared at 1 h / 2 h / 4 h	74 / 99 / 138	51 / 90 / 131
Levels cleared at the end	143	138
Median a_l/h_l on cleared levels	0.77	**0.68**
Mean public score	59.4	**63.5**
Prompt tokens from cache	98.4%	93.3%

Table 8 | The two full runs nearest to a perception ablation. They differ in more than the briefing, so the comparison is suggestive only. Level counts are from each run's own log of clear times.

D Open-Source Artifacts and Reproducibility

Licence. All code we wrote, the model-serving scripts, the evaluation harness and the experiment logs are public under the MIT-0 licence, a public-domain-equivalent licence as the competition rules require, at https://github.com/Harsidak/SARBLOH. Third-party components keep their own licences and are listed in the repository's `NOTICE` file.

Kaggle notebook and submission. The official scored benchmark evaluation was submitted directly from the public Kaggle competition notebook available at https://www.kaggle.com/code/banwait13/sarbloh-experimentation.

Run log. The paper names its two full public runs by configuration. In the experiment log, Run A (briefing off) is the run of 23 October 2026 and Run B (briefing on) the run of 27 October 2026, as in Table 8.

Scripts and evaluation reproduction. The figures, and the numbers they plot, are produced by reproducible scripts in the released repository, under `Writing/scripts/` (with input tables in `Writing/data/`):

- `collect_data.py`: parses submission logs, per-game score tables, and perception rubrics into `Writing/data/`.
- `extract_curves.py`: extracts levels cleared over wall-clock time from raw run logs.
- `error_budget.py`: splits points lost in a run into unreached levels, attempted levels, and action overhead under official RHAE weights.
- `make_perception_figure.py`: runs the perception engine on real game frames to render objects, maps, and briefings.
- `make_briefing_budget.py`: generates the two `tr87` briefings of Figure 17 using the agent's `Briefing` module.
- `make_method_figures.py`: renders Method and Results figures from run telemetry.
- `plot_results.py`: renders benchmark curves and evaluation figures as vector PDF and PNG.